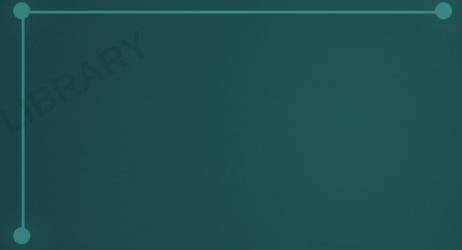
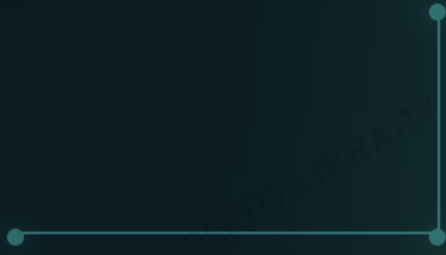
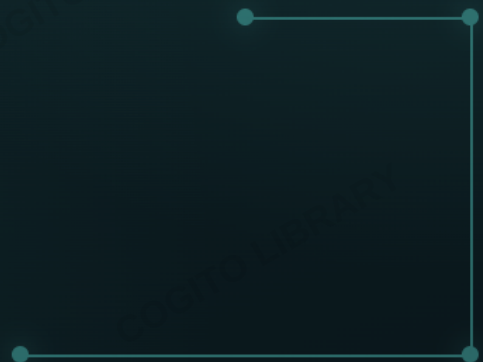
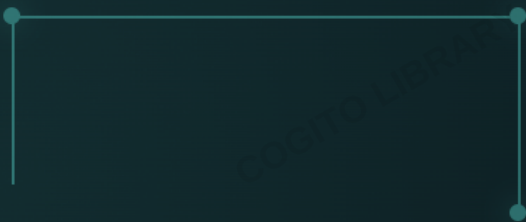


H A R N E S S D E S I G N



하네스 엔지니어링 실전 플레이북

14개 파이프라인에 6원칙을 적용한
1인 개발자의 기록



TEAM GRIT

COGITO LIBRARY

프롤로그 · 내 파이프라인은 몇 점일까요

지금 당신이 운영 중인 AI 파이프라인은 몇 점일까요?

아마 이 질문에 숫자로 답하기 어려우실 겁니다. 블로그 초안 생성, 뉴스 큐레이션, 카드뉴스 이미지, SNS 스톱, 밈 발굴, 멘토링 리뷰, 시드 리뷰. 어느 순간 여덟 개, 열두 개, 스무 개로 늘어나 있습니다. 그 중 몇 개가 잘 돌아가고 있는지 물으면, 대부분의 답은 이렇게 돌아옵니다.

“에러는 안 나니까 괜찮겠지.”

저도 그 문장으로 2년을 살았습니다. 스물세 개의 파이프라인이 매일 밤 돌아갔고, 그 중 다섯 개는 제 밤 시간의 70%를 먹어치웠습니다. 새벽 세 시에 Slack 알림이 울리면 “또 그 파이프라인이구나” 하고 재시작 버튼을 눌렀습니다. 실패한 적은 한 번도 없습니다. 다만 잘 돌아간 적도 한 번도 없었습니다. 저는 그 두 상태를 구분하지 못했습니다.

그 사실을 알게 된 날이 있습니다. 평범한 화요일 새벽, 스프레드시트 하나를 열었습니다. 가로축에 여섯 개의 원칙을 적고, 세로축에 스물세 개의 파이프라인 이름을 적었습니다. 각 셀에 1에서 5점을 채워 넣고 가중치를 곱해서 합산했습니다. 30분 뒤 마지막 열의 평균 셀에 숫자가 떴습니다.

4.82.

좋은 점수였을까요, 나쁜 점수였을까요? 그날 새벽 세 시에 저는 처음으로 그 질문에 답할 수 있었습니다. 23개 중 S 등급은 단 한 개. A 등급이 여섯 개. 가장 많은 건 B 등급 열 개. 그리고 제 밤을 먹어치우던 다섯 개는 전부 C 등급이었습니다. 제가 가장 힘들었던 파이프라인이 가장 낮은 점수를 가진 파이프라인이었습니다. 주관과 객관이 드디어 만났던 순간이었습니다.

이 책은 그 스프레드시트를 당신의 것으로 만드는 책입니다.

여섯 개의 원칙은 Anthropic Labs에서 나왔습니다. 스물세 개의 파이프라인은 제가 실제로 운영하는 것들입니다. 두 개를 곱한 점수표가 이 책의 심장입니다. 심장을 지탱하는 뼈대는 네 가지입니다. 생성과 평가를 분리하는 법. 검증과 루브릭을 쌍으로 쓰는 법. 에이전트 간 통신을 파일로 옮기는 법. 컨텍스트를 줄이고 신선도를 감사하는 법. 각각은 한 챕터씩. 네 챕터가 끝나면 당신은 앞의 두 권이 남겨둔 구멍, 측정의 공백을 드디어 메울 수 있게 됩니다.

앞의 두 권이란 같은 3부작에 속하는 『팀이 필요하다. 사람 말고.』와 『Claude Code로 1인 개발팀 만들기』입니다. 첫 권이 왜 하네스를 만들어야 하는지를 말했고, 둘째 권이 어떻게 만드는지를 말했습니다. 이 책은 마지막 한 글자, 몇 점인지를 말합니다. 세 권을 반드시 순서대로 읽어야 하는 것은 아니지만, 이 책을 펴신 순간 앞의 두

권의 전제는 이미 이해하고 계신다고 가정합니다. 그 가정 덕분에 이 책의 지면은 사상이 아니라 실측에 집중할 수 있습니다.

저는 새벽 세 시의 그 스프레드시트 덕분에 몇 달 만에 평균을 5.06까지 끌어올렸습니다. 4.82에서 5.06. 작아 보이는 차이인데도 그 숫자가 이 책의 자랑거리가 되는 이유가 있습니다. 제가 그 숫자가 어디서 왔는지 정확히 안다는 것입니다. 어느 원칙이 올랐고, 어느 원칙은 여전히 낮은지. 다음 달에 어디를 건드려야 하는지.

모델은 곧 바뀝니다. 3개월 뒤에는 오늘 쓰는 모델보다 더 좋은 게 나옵니다. 그런데 제 스프레드시트의 여섯 개 축은 그대로 남습니다. 새 모델에 같은 점수를 매겨서 비교하면 됩니다. 제가 쌓은 것은 모델이 아니라 점수표였습니다.

이 책은 당신의 파이프라인에 점수가 쓰이게 되는 책입니다. 점수가 쓰이면 방향이 보입니다. 방향이 보이면 밤 세 시에 재시작 버튼을 누르는 대신 다음 주에 고칠 한 가지가 정해집니다. 4.2점과 4.3점의 차이는 중요하지 않습니다. 점수가 있는 파이프라인과 점수가 없는 파이프라인의 차이가 전부입니다.

시작하겠습니다.

인용·참고

- Source: 저자의 파이프라인 운영 경험 (가상 도메인 Quiet Ark / PageMark / DeckBrew / Nightwatch 로 재구성)
- Source: 앞의 두 권 『팀이 필요하다. 사람 말고.』 『Claude Code로 1인 개발팀 만들기』 — 본서는 MEASURE 축의 3부작 완결

Ch 1 · 측정하지 않는 파이프라인은 존재하지 않습니다

0. 이 장의 문장 하나

이 책이 시리즈의 세 번째 권인 이유가 이 장에 있습니다. 앞의 두 권이 하네스를 왜 만들고 어떻게 만드느냐를 끝냈다면, 이 권은 만든 하네스를 몇 점으로 측정하는 것입니다. 그 공백을 메우지 않으면 앞서 만든 모든 것이 “잘 돌아가는 것 같다”라는 착각의 연료가 됩니다.

1. 3부작에서 이 권이 맡는 자리

책	축	다루는 질문
『팀이 필요하다. 사람 말고.』	WHY	왜 하네스를 만들어야 하는가
『Claude Code로 1인 개발 팀 만들기』	HOW	하네스를 어떻게 만드느냐
『하네스 엔지니어링 실전 플레이북』 (이 책)	MEASURE	만든 하네스가 몇 점인가

세 권을 반드시 순서대로 읽을 필요는 없습니다. 다만 이 책을 펴신 순간 앞의 두 권의 결론은 이미 받아들이고 계신다고 가정하겠습니다. 그 가정 덕분에 이 책은 사상을 다시 설명하지 않고 실측에 집중할 수 있습니다.

재설명의 유혹은 무섭습니다. “하네스란 무엇인가”를 한 번 더 다루면 독자가 더 친절하게 느낄 것 같습니다. 하지만 그 한 번 더가 이 책의 실측 공간을 30페이지 먹어치웁니다. 이 책은 크로스레퍼 원칙을 지킵니다. 필요하다면 앞 두 권의 해당 챕터 번호를 가리킵니다. 재설명은 하지 않습니다.

2. 왜 MEASURE가 공백이었나

앞의 두 권이 미처 다루지 못한 것은 게으름이 아닙니다. 측정은 WHY와 HOW와 다른 층위이기 때문입니다. WHY가 사상이고 HOW가 구현이라면, MEASURE는 운영입니다. 운영은 구현이 끝난 뒤에 발생합니다. 파이프라인이 실제로 돌아가는 시간 위에서만 점수를 매길 수 있습니다.

점수를 매기려면 다음이 필요합니다.

- 측의 이름 (여섯 개 — 이 책 2장)
- 측의 가중치 (여섯 개 × 가중치 — 이 책 2장)
- 1~5점의 구체적 장면 (각 측 × 5 = 30개 이상 — 이 책 3~6장)
- 측정 대상 파이프라인의 목록 (최소 세 개)
- 한 번 측정한 뒤 석 달 뒤 재측정하겠다는 약속

이 다섯 가지가 마련돼야 측정이 시작됩니다. 앞 두 권이 마지막 두 가지를 다루지 않은 건 자연스럽습니다. 독자 각자의 파이프라인 목록과 재측정 캘린더는 저자가 대신 써 줄 수 없기 때문입니다. 앞의 세 가지도 운영 경험 없이는 구체 장면이 비어 있어 서술이 막힙니다. 이 책이 세 번째 권인 이유가 바로 이 결핍에 있습니다.

3. “잘 돌아가는 것 같다” 라는 말의 정체

운영 중인 파이프라인이 에러를 내지 않는다는 사실에서 잘 돌아간다고 도약하는 순간이 있습니다. 이 도약이 누적되면 스무 개의 파이프라인 중 실제로 잘 돌아가는 것이 몇 개인지 아무도 모르게 됩니다. 저도 그 상태로 2년을 살았습니다.

“잘 돌아가는 것 같다”의 실체를 세 갈래로 분해할 수 있습니다.

3-1. 사일런트 저하

에러는 없지만 품질이 서서히 내려가는 상태. 대개 원인은 P6(Staleness) 위반입니다. 3개월 전 프롬프트가 아직 살아 있고, 모델은 업데이트됐고, 브랜드 가이드는 바뀌었습니다. 6장에서 다룰 단종된 모델명이 3개월 방치된 파이프라인 이 이 갈래의 전형입니다.

3-2. 재생성 루프 폭발

합격률은 높아 보이는데 평균 API 호출이 늘고 있는 상태. 대개 원인은 P1(Gen/Eval) 위반입니다. 평가자가 자기 자신이라 피드백이 구체적이지 않고, 재생성이 같은 약점을 반복합니다. 3장의 128 → 8 사례가 이것입니다.

3-3. 허수 채점

자기 점수는 계속 높는데 외부인이 보면 납득이 안 되는 상태. 대개 원인은 P3+P5 공백입니다. validate도 rubric도 없어서 “좋아 보인다”가 그대로 점수가 됩니다. 4장의 DeckBrew Card News Image 사례가 이것입니다.

세 갈래 중 어느 것이든 공통 증상은 운영자가 불안하다는 것입니다. 파이프라인이 돌고 있는데 자꾸 확인하고 싶어집니다. 확인하면 불안이 잠시 가라앉고, 몇 시간 뒤 다시 확인하게 됩니다. 자꾸 확인하고 싶어지는 건 파이프라인이 당신에게 점수를 주지 않기 때문입니다. 숫자가 없으니 안심할 근거도 없습니다.

4. 자가진단 8문항

다음 여덟 문항에 답하세요. 다 큰 소리로 “아니요”라고 말할 수 있다면 본 책이 당신에게 가르칠 건 많지 않습니다. 절반 이상 “예”라면 정확히 본서의 주인공이 당신입니다.

- ☐ 운영 중인 AI 파이프라인이 세 개 이상이다.
- ☐ 그 중 몇 개가 잘 돌아가는지 숫자로 답할 수 없다.
- ☐ 최근 한 달 안에 재생성 루프로 API 호출이 예상보다 10배 이상 나온 적이 있다.
- ☐ Generator와 Evaluator가 같은 세션(또는 같은 프롬프트)에서 동작한다.
- ☐ 파이프라인의 “좋은 결과” 기준이 문서로 존재하지 않는다.
- ☐ 파이프라인이 실패했을 때 어디서 실패했는지 재현하려면 로그를 뒤져야 한다.
- ☐ “컨텍스트를 더 넣으면 좋아진다”고 믿고 있다.
- ☐ 파이프라인의 프롬프트·지침 파일을 3개월 이상 안 열었다.

네 개 이상 “예”라면 본 책의 2장 이후가 당신을 위한 페이지입니다. 여덟 개 전부 “예”라면 본 책 한 권만으로는 부족합니다 — 3부작의 앞 두 권부터 다시 보셔도 늦지 않습니다.

5. 측정의 반례 — 점수를 매겨서 더 나빠진 적이 있는가

“측정하는 순간 품질이 보장된다”는 주장은 과장입니다. 측정이 오히려 일을 망치는 경우가 있습니다.

5-1. 측정을 목표로 오인

점수가 보상의 직접 대상이 되면 운영자는 점수를 올리기 위해 움직입니다. 품질은 올리지 않고 숫자만 올리는 경로가 생깁니다. 루브릭 가중치를 자신에게 유리하게 바꾸거나, 평가자의 기준을 몰래 낮춥니다. 이것은 점수의 본질 훼손입니다.

방지책: 점수는 진단 도구지 보상 도구가 아닙니다. 점수가 오르면 좋은 일이지만, 오르지 않는다고 해서 벌 주지 않습니다. 점수는 다음 달 어디를 건드릴지를 보여줄 뿐입니다.

5-2. 너무 자주 측정

측정 주기가 일주일이면 평가 노이즈가 신호를 덮습니다. 주간 변동이 월간 변동보다 큰 이상한 그래프가 나옵니다. 평가자도 피로해집니다.

방지책: 분기(3개월) 주기가 적합합니다. 매주 측정할 필요는 없습니다. 점수의 움직임은 주 단위가 아니라 분기 단위로 봐야 의미가 있습니다. 분기 끝에 재측정하고 직전 분기와의 차이를 보는 것이 이 책이 권하는 리듬입니다.

5-3. 모든 축을 동시에 건드리기

점수가 낮다고 여섯 개 축을 한 번에 고치려 들면, 결과가 나왔을 때 어느 변화가 어느 개선을 만들었는지 귀속시킬 수 없습니다. 실전 개선 리듬은 의도적으로 한 번에 2~3축만 건드립니다.

방지책: 가중치 순서로, $P1 > P3 > P2 \cdot P4 \cdot P6 > P5$. 큰 것부터.

5-D. 두 해 동안의 자기 기록 · 왜 이 책이 지금 나왔는가

저자 개인의 기록 하나를 짧게 공유합니다. 2024년 초에 첫 파이프라인을 운영하기 시작했을 때, 저는 여기서 여러 번 언급한 “잘 돌아가는 것 같다”의 상태에 있었습니다. 에러가 없으니 괜찮다고 생각했습니다. 하지만 매

일 밤 두 시간씩 뭔가 이상한데 라는 불안으로 파이프라인을 다시 열어 보곤 했습니다. 그 불안이 어디서 오는지 몰랐습니다.

2025년 중반에 Rajasekaran 논문 초안이 Anthropic Labs 내부에서 공유되고 있던 것을 뒤늦게 접했습니다. 2026년 3월에 공식 발표된 그 논문이, 이미 몇 달 전부터 현업 파이프라인 운영자들 사이에서 회자되던 내용이었습니다. 읽었을 때 가장 먼저 느낀 건 해방감이었습니다. 내 불안에 이름이 있었다. 원칙 여섯 개 각각에 이름이 붙어 있으니 “아, 내가 매일 불안했던 건 P1이 뚫려 있었기 때문이구나”라고 언어화가 됐습니다. 언어화가 되니 해결 방법도 구조적으로 보였습니다.

그 다음 반년간 저는 스물세 개 파이프라인 전체를 차례로 개선했습니다. 처음 여섯 개 파이프라인을 고쳤을 때, 이 과정을 다른 누군가에게도 전해야 하지 않을까라는 생각이 들었습니다. 3부작의 앞 두 권은 그래서 나왔습니다. 그런데 두 권을 내고 나서도 축정의 공백이 남아 있다는 독자 피드백이 계속 들어왔습니다. “이걸 이만큼만 들었는데 좋은 건가요?”라는 질문. 그 질문에 답을 주기 위한 책이 지금 당신 손에 있는 이 책입니다.

이 책이 나온 시점이 정확히 저의 23개 파이프라인 평균이 5.06인 시점입니다. 100점 만점이라면 D-tier 상위에서 B-tier 중위로 올라온 수준. 완벽이라 할 수 없고, 저도 여전히 C-tier 파이프라인 두 개를 가지고 있습니다. 하지만 지금 어디가 약한지 정확히 안다는 상태가 되었습니다. 이 상태를 당신에게 전하는 것이 이 책의 목적입니다. 완벽이 아니라 자기 파이프라인의 상태를 숫자로 아는 상태.

6. 앞으로 이 책의 지도

2장에서 여섯 개 축을 정의하고, 3~6장에서 실전 네 축을 한 챕터씩 심화합니다.

- Ch 2 · 6원칙 — 점수표의 축 (정의)
- Ch 3 · P1 Gen/Eval 분리 (실전 1 · 가중치 최고)
- Ch 4 · P3+P5 검증 계약과 가치 인코딩 (실전 2 · 쌍으로)
- Ch 5 · P4 파일 기반 핸드오프 (실전 3)
- Ch 6 · P2+P6 컨텍스트 불안과 노후화 감사 (실전 4 · 쌍으로)
- Ch 7 · 스코어카드 시스템 — 14 파이프라인 전수 공개 (심장부)
- Ch 8 · 실패 로그 (스케일 1)
- Ch 9 · 에필로그 (스케일 2)

진단 두 장, 정의 한 장, 실전 다섯 장, 스케일 두 장. 그리고 부록 다섯 종. 책을 덮기 전에 부록 A의 빈 스코어카드에 당신 파이프라인 세 개를 채워 넣는 것이 이 책의 변화입니다.

7. 내일 30분에 할 것

- ☐ 지금 운영 중인 AI 파이프라인을 한 줄씩 적어 목록을 만드세요. 대충이라도.
 - ☐ 각 파이프라인 옆에 운영 기간과 체감 품질을 1줄씩 메모.
 - ☐ 적고 난 목록을 보세요. 숫자가 하나도 없을 것입니다. 그 사실 자체가 이 책의 출발점입니다.
 - ☐ 목록에서 가장 의심스러운 파이프라인을 표시해 두세요. 2장 끝의 30분 블록에서 이 파이프라인부터 점수 매깁니다.
-

인용·참고

- Source: 저자의 3부작 중 1·2권 『팀이 필요하다. 사람 말고.』 『Claude Code로 1인 개발팀 만들기』 — WHY·HOW 축의 선행서.
- Source: 저자의 운영 스튜디오 Quiet Ark의 스물세 개 파이프라인 2년 운영 경험.

Ch 2 · 6원칙 — 점수표의 축

0. 이 장의 문장 하나

당신이 지금 운영 중인 파이프라인은 6개의 축으로 점수가 매겨질 수 있습니다. 축의 이름을 모르면 점수도 없습니다. 이 장은 당신이 앞으로 내내 돌아오게 될 책 전체의 참조표입니다.

1. 논문은 있는데 숫자는 없는 이유

Anthropic Labs의 Prithvi Rajasekaran이 2026년 3월 24일에 공개한 논문이 있습니다. 제목은 Harness design for long-running application development. 지면이 긴 편이 아니라 읽으려면 30~40분이면 충분합니다. 이미 읽으셨을 수도 있습니다. 읽으셨다면 이런 장면을 기억하실 겁니다.

“Even a frontier coding model like Opus 4.5... will fall short of building a production-quality web app if given only a high-level prompt.”

— P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24

최전선 모델조차 고수준 프롬프트만으로는 프로덕션 품질 웹앱을 못 만든다. 이 한 문장이 왜 무거운지는 앞의 두 권이 이미 설명했습니다. 모델이 아니라 모델 주변의 구조물이 품질을 결정한다는 사상. 그 구조물을 우리는 하네스라고 부릅니다.

그런데 논문을 덮고 당신의 파이프라인 앞으로 돌아오면 질문이 바뀝니다.

“그래서 내 하네스는 어느 정도로 좋은가?”

논문에는 그 답이 없습니다. 논문은 어떻게 만들라까지 말했지만 얼마나 좋은지 점수 매기는 법을 주지는 않았습니다. 숫자가 없으니 리뷰 회의 때마다 “이 파이프라인 괜찮아 보여요” 와 “글쎄요 뭔가 불안해요” 가 대립합니다. 주관과 주관의 충돌. 이 책은 그 충돌을 끊기 위한 것입니다.

주관을 끄는 방법은 하나입니다. 축을 정하고 점수를 매기는 것. 축은 여섯 개. 점수는 1~5. 축마다 가중치. 이 셋만 있으면 파이프라인마다 하나의 숫자가 나옵니다. 그 숫자를 티어로 묶으면 S, A, B, C, D 등급이 됩니다. 제가 정한 것이 아닙니다. Rajasekaran 논문과 그 선행 포스트에서 원칙의 이름이 이미 주어졌고, 저자의 운영 스튜디오 Quiet Ark는 그 이름들에 운영 가중치를 붙였습니다. 이 책은 그 가중치 붙은 6축을 그대로 씁니다.

2. 여섯 개 축 · 이름·가중치·한 줄

먼저 표부터 드리겠습니다. 이 표는 앞으로 매 챕터 첫 머리에서 다시 만나게 될 참조표입니다.

ID	원칙	가중치	한 줄 정의
P1	Generator-Evaluator Separation	0.25	만든 놈이 평가하면 안 됩니다
P2	Context Anxiety Mitigation	0.15	세션을 강제로 쪼개세요. 안 쪼개면 품질이 녹습니다
P3	Sprint Contract (Verification Contract)	0.20	시작 전에 성공이 무엇인지를 파일로 써 두세요
P4	File-Based Handoff	0.15	에이전트 간 통신은 대화가 아니라 파일입니다
P5	Grading Criteria Value Encoding	0.10	좋은 결과를 수치화된 루브릭으로 못 박으세요
P6	Harness Staleness Audit	0.15	3개월 전에 쓴 프롬프트가 지금도 맞는지 주기 감사하세요
합계		1.00	

가중합 공식은 이렇게 됩니다.

$$\text{가중합} = P1 \times 0.25 + P2 \times 0.15 + P3 \times 0.20 + P4 \times 0.15 + P5 \times 0.10 + P6 \times 0.15$$

각 P에 1~5점을 넣으면 가중합은 1.00에서 5.00 사이의 숫자가 됩니다. 그걸 티어로 묶습니다.

- **S-tier**: 4.0 이상 5.0 이하
- **A-tier**: 3.0 이상 4.0 미만
- **B-tier**: 2.0 이상 3.0 미만
- **C-tier**: 1.0 이상 2.0 미만
- **D-tier**: 1.0 미만

대부분의 운영 중인 파이프라인은 B와 C 사이입니다. 저의 평균도 첫 측정 때 그랬습니다. S-tier는 23개 중 1개였고, 그 한 개가 의외의 파이프라인이었다는 얘기는 7장에서 하겠습니다.

3. 각 원칙의 ‘안 지키면 벌어지는 것’

여섯 개 축 각각이 깨졌을 때 현장에서 무엇이 터지는가를 한 줄씩 먼저 보여드리겠습니다. 상세는 Ch3~6에서 한 챕터씩 다룹니다.

P1 — 만든 놈이 평가하면 안 됩니다

같은 세션이 콘텐츠를 생성하고 그 결과를 자체 평가할 때 나오는 점수는 언제나 높습니다. 자기가 만든 것을 자기가 채점하니까요. 가장 눈에 띄는 증상은 재생성 루프입니다. 평가자가 관대하니 합격이 많고, 어쩌다 한 번 불합격이 나오면 같은 세션이 같은 방식으로 다시 만들고 다시 합격을 줍니다. 그 사이 API 호출은 8회에서 128회로 폭발합니다. 비용이 하루 \$3에서 \$45로 뛰어도 점수표에는 4.5점이 적혀 있습니다.

P2 — 컨텍스트 불안과 싸우세요

“컨텍스트는 많을수록 좋다”는 말은 함정입니다. 8,000 토큰을 넣은 파이프라인이 3,000 토큰으로 줄이자 품질이 올라간 실측이 있습니다. 맥락이 많으면 안심이 되지만, 모델 입장에서는 무엇을 봐야 할지 흐려집니다. 세션이 50턴을 넘기면 초기 지침은 모델의 작업 기억에서 밀려 나갑니다. 강제로 쪼개지 않으면 동일한 작업을 다시 돌렸을 때 결과가 30% 이상 벌어집니다.

P3 — 시작 전에 계약서를 쓰세요

“되는 것 같다”로 파이프라인을 출시하면 2주 뒤에 깨진 콘텐츠가 발견됩니다. 독자는 그 2주 사이에 이미 불신을 누적했습니다. `validate.py` 하나만 있으면 배포 직전에 JSON 스키마, 길이 범위, 금칙어, 품질 점수 네 가

지를 자동 검사합니다. 파이프라인이 아니라 세션인 것과 진짜 파이프라인의 차이는 이 파일 하나에서 갈립니다.

P4 — 파일로 말하세요

에이전트끼리 대화로 일하면 어느 날 세션이 OOM으로 죽습니다. 12시간짜리 작업이 6시간 지점에서 멈춘 순간, 대화형이라면 처음부터 재시작입니다. 파일 기반이었다면 마지막 `output.json` 에서 재개하면 됩니다. 파일은 복구 지점이자 인터페이스입니다. 파일로 적히지 않은 통신은 파이프라인의 구멍입니다.

P5 — 좋다는 게 뭔지 써 두세요

리뷰 회의마다 “이 블로그 톤 좋아 보이는데” 와 “글쎄 뭐가 좀 부족한데” 가 싸웁니다. 둘 다 맞고 둘 다 틀립니다. 좋다는 기준이 파일로 없기 때문입니다. `rubric.yaml` 에 7축을 1~5점 기준으로 못 박아 두면 다음 리뷰 회의는 15분에 끝납니다. 토론이 아니라 점수 매기기가 됩니다.

P6 — 신선도를 감사하세요

3개월 전에 쓴 시스템 프롬프트에 단종된 모델명이 들어 있었습니다. 에러는 안 났습니다. SDK가 silently 최신 모델로 fallback했고, 파이프라인은 계속 돌았습니다. 그 사이 품질이 조용히 내려갔습니다. 우리는 석 달 뒤 rubric 점수가 미세하게 떨어진 것을 보고서야 원인을 찾았습니다. 에러가 나지 않는 하락이 가장 위험합니다.

4. 왜 P1의 가중치가 가장 무거운가

표를 다시 보시면 가중치가 균일하지 않습니다. P1이 0.25로 가장 무겁고, P5가 0.10으로 가장 가볍습니다. 이 불균형은 우연이 아닙니다.

P1(Gen-Eval 분리)이 깨지면 나머지 모든 원칙의 점수가 거짓으로 보입니다. 자기 채점이 4.5점을 찍고 있으면 P3 검증 계약도 “통과하고 있는 것처럼” 보이고, P5 루브릭도 “잘 작동하는 것처럼” 보입니다. 평가자가 관대하기 때문입니다. 그래서 P1은 모든 원칙의 진실성을 담보하는 축이고, 가중치가 0.25입니다. 책 전체에서 P1이 먼저 다뤄지는 이유도 같습니다. Ch30이 P1입니다.

P3(Sprint Contract)이 0.20인 이유는 그 다음 무거운 비용이 이 지점에서 발생하기 때문입니다. 계약 없이 돌아가는 파이프라인은 배포 이후에 비용을 치릅니다. 깨진 콘텐츠를 본 독자의 신뢰, 그걸 되돌리는 재작업 시간. 이 비용은 눈에 안 보이게 커져서 P3을 두 번째 무게로 둡니다.

P5(Value Encoding)가 0.10으로 가장 가벼운 이유가 의외일 수 있습니다. 루브릭이 덜 중요해서가 아닙니다. 루브릭이 없어도 P1·P3이 돌고 있으면 최소 품질은 나옵니다. 루브릭은 그 위에 얹히는 가치 층입니다. 기초가 없는데 루브릭부터 만드는 건 순가락 없이 국을 만드는 일과 비슷합니다. 그래서 맨 마지막 층이고, 가중치가 가장 가볍습니다.

P2·P4·P6은 0.15로 균등합니다. 이 세 원칙은 파이프라인의 지속 가능성을 담당합니다. 컨텍스트가 깨끗한지, 통신이 파일인지, 프롬프트가 신성한지. 이것들은 세 달에 한 번씩 점수가 조용히 떨어지는 지점들입니다. 균등한 가중치를 준 이유는 어느 하나만 낮아도 충분히 아프기 때문입니다.

이 가중치 배분에 정답이 있는 것은 아닙니다. 당신의 도메인에서는 P2가 P1보다 중요할 수도 있습니다. 그 경우 이 책의 가중치 표를 당신의 가중치 표로 바꾸시기 바랍니다. 숫자는 제안이지 명령이 아닙니다. 다만 어느 축이 몇 점인지는 바뀌어도 바뀌지 않습니다.

5. 왜 우리 번호인가 — 출처 경계 박스

본 책의 6원칙 번호(P1~P6)는 저자가 Quiet Ark 스튜디오에서 Rajasekaran 논문(Harness design for long-running application development, 2026-03-24)과 그 선행 포스트(Effective harnesses for long-running agents)의 원칙들을 실전 적용을 거쳐 정립한 번호입니다.

논문 원문을 찾아 대조하시면 원칙의 이름은 일치하지만 번호와 가중치는 저자 것이 아님을 확인하실 수 있습니다. 원문에서는 가중치가 명시되지 않았고, 번호도 순서가 다릅니다. 이 책은 원칙의 이름을 빌려 오되, 점수표라는 운영 도구를 위해 가중치를 붙였습니다.

원 논문을 찾으신다면 번호가 아니라 이름으로 대조하시기 바랍니다. 이름은 여섯 개 그대로입니다. Generator-Evaluator Separation. Context Anxiety Mitigation. Sprint Contract (Verification Contract). File-Based Handoff. Grading Criteria Value Encoding. Harness Staleness Audit.

이 경계선을 분명히 해 두는 이유는 독자가 논문을 직접 찾아봤을 때 번호가 달라서 혼란을 겪지 않게 하기 위함입니다. 번호는 운영 도구. 이름은 논문.

5-B. 왜 가중치가 균등하지 않은가 · 다른 가중치 설계 3안

저자의 스튜디오가 쓰는 가중치는 0.25/0.15/0.20/0.15/0.10/0.15입니다. 이 배분은 절대값이 아니라 이 스튜디오의 운영 경험에서 맞춰진 값입니다. 다른 운영 환경에서는 다른 배분이 나올 수 있습니다.

비교를 위해 세 가지 대안 가중치 설계를 봅시다.

대안 A · 비용 민감 스튜디오 (API 예산이 빠듯한 환경)

P1 : 0.35 ← 재생성 루프 방지가 가장 급함
P2 : 0.10
P3 : 0.20
P4 : 0.10
P5 : 0.10
P6 : 0.15

P1을 0.35까지 끌어올리는 이유: 비용 폭발이 가장 아픈 환경에서는 Gen/Eval 분리가 곧 예산 생존이기 때문. 반대로 P2·P4는 나중에 고쳐도 큰 지장이 없음.

대안 B · 엔터프라이즈 품질 민감 스튜디오

P1 : 0.20
P2 : 0.15
P3 : 0.20
P4 : 0.15
P5 : 0.20 ← 루브릭이 핵심
P6 : 0.10

P5를 0.10에서 0.20으로 두 배. 엔터프라이즈 환경에서는 품질의 주관적 공유 기준이 외부 감사·보고의 근거가 되므로 루브릭이 본거지. 재생성 루프는 상대적으로 비용이 덜 아픈 환경이라 P1은 표준치.

대안 C · 1인 크리에이터 환경

P1 : 0.20
P2 : 0.20 ← 컨텍스트 피로가 가장 큼
P3 : 0.15
P4 : 0.10
P5 : 0.15
P6 : 0.20 ← 혼자 운영하면 노후화 방지 위험 큼

P2와 P6에 각각 0.20. 1인 크리에이터는 모든 파이프라인을 한 머리로 돌리기 때문에 컨텍스트 불안이 다른 환경보다 훨씬 큼. P6도 같은 이유로 높음. 혼자 운영하면 자기 파이프라인이 노후화되는 것을 자기가 감지하기 어렵기 때문.

당신의 환경은 어디에 가까운가요? 가중치를 조정하는 것은 책의 원칙을 위반하는 게 아니라 원칙을 자기 상황에 맞게 재적용하는 것입니다. 단, 한 번 조정했으면 그 분기 동안은 바꾸지 말 것. 가중치가 자주 바뀌면 분기 간 비교가 의미 없어집니다.

6. 이 책의 규칙 — 어느 챕터가 어느 원칙을 다루는가

앞으로 여섯 개 축 각각이 챕터를 연습니다. 순서는 가중치가 아니라 실전에서 먼저 부딪히는 순서입니다.

- Ch 3 · P1 Gen/Eval 분리 — 가장 무거운 축, 다른 모든 점수의 진실성을 담보
- Ch 4 · P3 + P5 검증 계약과 가치 인코딩 — validate와 rubric은 쌍으로 쓰여야 함
- Ch 5 · P4 파일 기반 핸드오프 — 대화가 아니라 파일
- Ch 6 · P2 + P6 컨텍스트 불안과 노후화 감사 — 조용한 품질 하락의 두 얼굴

네 챕터가 끝나면 Ch 7이 옵니다. Ch 7은 이 여섯 축과 제 실측 14개 파이프라인을 전부 곱해서 점수표를 보여 드리는 챕터입니다. 이 책의 심장부입니다. 그 심장부가 뛰려면 먼저 여섯 축의 뜻을 알아야 하기에 Ch2의 자리가 여기입니다.

Ch 8은 점수가 내려간 곳에서 무엇이 터졌는가를 5건의 실패 로그로 기록합니다. 앞의 네 챕터 + Ch 7을 읽으신 독자가 비로소 실패를 원칙 번호로 귀속시킬 수 있게 되는 지점입니다.

이것이 이 책의 골격입니다. 지금부터는 축마다 한 챕터씩. 숫자가 뒤따를 겁니다.

7. 내일 30분에 할 것

- ☐ 본 챕터의 §2 표(P1~P6 이름·가중치·한 줄 정의)를 손으로 한 번 그리세요. 타이핑 말고 종이에.
- ☐ 당신이 운영 중인 파이프라인 중 한 개를 고르세요. 가장 오래된 것 혹은 가장 자주 돌아가는 것.
- ☐ 그 파이프라인에 P1~P6을 1~5점으로 채우세요. 확신 없으면 3점.

☐ 가중합을 계산하세요.

☐ 그 숫자가 본 책에서 당신이 처음으로 얻게 되는 당신 것의 숫자입니다. 책을 덮기 전에 잊지 마세요.

인용·참고

- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24.
- Source: Anthropic Engineering, Effective harnesses for long-running agents (선행 포스트).
- Source: 저자의 운영 스튜디오 Quiet Ark에서 정립한 6원칙 번호·가중치 (Rajasekaran 논문 재해석, 실전 운영 기반).
- Source: 저자의 파이프라인 운영 경험 (가상 도메인 Quiet Ark / PageMark / DeckBrew / Nightwatch 로 재구성).

Ch 3 · P1 Gen/Eval 분리 — 만든 놈이 평가하면 안 됩니다

0. 이 장의 문장 하나

만든 사람이 자기 작품을 채점하면 점수가 높습니다. 그 점수가 실제 품질과 같은지는 별개 문제입니다. 이 단순한 사실 하나에서 여섯 개 축 중 가중치가 가장 무거운 P1이 시작됩니다.

1. 왜 P1이 가장 무거운가 — 한 번 더

2장에서 P1의 가중치가 0.25인 이유를 짚었습니다. 진실성 담보. 여기서 한 번 더 짚는 이유는, 다른 네 원칙을 읽을 때마다 P1이 뚫려 있으면 나머지 점수가 거짓으로 보인다는 사실을 계속 기억해 두어야 하기 때문입니다.

생각해 봅시다. 당신의 파이프라인에 `validate.py`가 있고(P3), 루브릭이 있고(P5), 파일 핸드오프가 잘 되어 있다고(P4) 가정하겠습니다. 가중합으로 5.5점쯤 나올 겁니다. 그런데 Generator와 Evaluator가 같은 세션입니다. `rubric.yaml`에 “hook 점수 5점”이 찍히는 이유가 “이 콘텐츠를 만든 LLM이 자기 콘텐츠에 5점을 주었기 때문”이라면, 그 5.5점은 실제 품질이 아니라 자기 채점의 관대함이 만든 숫자입니다.

외부 평가자가 다시 매기면 3.2점이 나옵니다. 이 간극이 반복되면 운영자는 점점 숫자를 믿지 않게 됩니다. 그러면 애초에 왜 점수를 매기는지 이유가 사라집니다. P1은 스코어카드 자체의 신뢰성입니다. 다른 원칙이 구조물이라면 P1은 그 구조물 아래의 기반입니다.

2. 실측 데이터 · 자기 점수 vs 외부 점수

네 개의 파이프라인에서 자기 점수와 외부 점수를 비교한 기록이 있습니다. 2026-04-17 시점 30건 샘플 기준.

파이프라인	자기 점수 평균	외부 점수 평균	간극
PageMark Blog Generation	4.3	3.1	-1.2
PageMark SNS/Threads	4.5	3.2	-1.3
DeckBrew Card News Image	4.2	3.4	-0.8
Quiet Ark Meme Generation	4.7	1.9	-2.8

네 개 파이프라인 전부에서 자기 점수가 외부 점수보다 높았습니다. 간극의 크기는 파이프라인마다 다른데, 눈여겨 볼 건 간극이 가장 큰 Quiet Ark Meme Generation이 7장에서 베이스라인 가중합 2.05였던 그 파이프라인입니다. P1 점수가 1점이었고, 자기/외부 간극이 2.8이었다는 것이 같은 사건의 다른 얼굴입니다.

자기 채점은 외부 채점보다 일관되게 높습니다. 높이의 정도가 하네스 약화의 지표가 됩니다. 외부 평가가 귀찮아서 안 하는 것이 아니라, 귀찮아서 안 하는 비용이 이 숫자입니다.

3. 재생성 루프의 비용 폭발

자기 점수가 높으니 대개는 평가가 “PASS”로 넘어갑니다. 그런데 가끔 자기 점수가 3점 미만으로 떨어지면 “REJECT” 후 재생성이 일어납니다. 재생성은 같은 프롬프트가 조금 흔들린 결과를 다시 만들고, 같은 평가자가 조금 흔들린 점수를 줍니다. 또 REJECT 나면 또 재생성. 이 루프가 폭발합니다.

DeckBrew News Curation이 Evaluator 분리 전에 겪었던 일입니다. 2026-02 기록.

항목	값
API 호출 / 완료 1건	128
평균 재시도	17회 (최대 34회)
비용 / 일	\$45
최종 산출물 품질	평균 3.4 (외부 재측정 시)

평가자가 관대해서 합격률이 높은 것처럼 보이는데, 어쩌다 한 번 엄격해진 날에 같은 세션이 같은 약점을 못 고친 채로 반복 생성하기 때문에 루프가 터집니다. 이 파이프라인이 가장 조용히 지갑을 태우고 있던 지점이었습
니다.

Evaluator를 분리하자 같은 지점에서 일어난 일.

항목	Before	After
API 호출 / 완료 1건	128	8
평균 재시도	17회	0.7회
비용 / 일	\$45	\$3
최종 산출물 품질	3.4	4.1

호출 수가 128에서 8로 떨어지면서 비용이 94% 감소했습니다. 주목할 건 품질까지 올랐다는 것입니다. Evaluator가 분리되자 “이번 초안은 이런 약점이 있습니다. 다시 쓰되 이 부분을 고치세요” 같은 지시가 담긴 피드백이 Generator에게 돌아가게 됐습니다. 같은 세션의 자기 평가는 이런 피드백을 만들 수 없습니다. 자기 약점을 모르니까요.

3-B. 재생성 루프의 분기별 로그

128회 호출이 어떻게 8회로 줄었는지를 주 단위 로그로 봅시다. 2026-02 한 달간의 실측.

주차	일 평균 API 호출 / 건	비용 / 일	조치	자기 점수	외부 점수
W1	128	\$45	— (이상 감지 전)	4.5	—
W2	132	\$47	이상 감지, 원인 분석	4.6	3.2
W3	45	\$15	Evaluator 별도 파일 분리	4.1	3.8

주차	일 평균 API 호출 / 건	비용 / 일	조치	자기 점수	외부 점수
W4	8	\$3	시스템 프롬프트 분리 + 세션 격리	—	4.1

주목할 지점 세 개.

첫째, W2에서 원인 분석이 시작됐을 때 자기 점수는 4.6으로 오히려 올라갔습니다. 평가자가 혼란스러워질수록 자기 방어적으로 관대해지는 패턴. 이게 자기 평가의 함정입니다.

둘째, W3에서 파일 분리만 했는데 호출 수가 128에서 45로 약 3배 감소했습니다. 같은 세션 안의 자기 평가 루프가 깨졌기 때문. 다만 시스템 프롬프트는 같았기 때문에 Evaluator가 여전히 Generator의 관점을 따라 하고 있어 점수 간극이 큼니다 (자기 4.1 vs 외부 3.8).

셋째, W4에서 시스템 프롬프트 분리 + 세션 격리를 하자 호출이 8까지 떨어지고 외부 점수가 4.1로 올라갔습니다. 진짜 분리가 일어난 시점. 품질이 올라간 것이 아니라 평가의 진실성이 회복된 것입니다. 128 → 8은 **94% 감소**가 아니라 **16분의 1 감소**. 재생성 루프가 증폭이 아니라 기하급수로 작동합니다.

3-C. 비용이 아닌 시간으로 환산하면

API 비용만 보면 하루 \$42 절감입니다. 하지만 이 파이프라인을 돌리는 Generator의 응답 대기 시간까지 합치면 환산이 달라집니다.

지표	Before (W1)	After (W4)	차이
건당 API 호출	128	8	16배
건당 처리 시간 (실측 분)	42	4	10배
일 처리 건수 (한 세션)	8	50	6배
일 비용	\$45	\$3	15배
운영자 모니터링 시간 (일)	60분	5분	12배

운영자가 이 파이프라인 때문에 낭비하던 시간이 하루 60분에서 5분으로 줄었습니다. 하네스는 비용 절감이 아니라 운영자 시간 회수의 문제입니다.

4. Generator와 Evaluator의 분리 3조건

“Evaluator를 분리했다”고 말하려면 세 조건이 다 맞아야 합니다. 하나라도 깨지면 분리한 것이 아닙니다.

조건 1 · 다른 파일

코드가 같은 파일 안에 있으면, 심지어 두 함수로 분리돼 있어도 조용히 상태를 공유합니다. 같은 모듈의 전역 변수, 같은 import된 프롬프트 상수, 같은 설정 dict. 조금만 게으르면 작성한 사람이 의도하지 않은 컨텍스트가 두 함수 사이를 건너뛰니다. 파일이 나뉘어야 그 가능성이 원천 봉쇄됩니다.

```
파이프라인 루트/  
├── generator.py    ← Generator가 여기에만  
├── evaluator.py    ← Evaluator가 여기에만  
├── prompts/  
│   ├── gen_system.md  
│   └── eval_system.md
```

조건 2 · 다른 세션

LLM 호출 단에서 세션 ID가 달라야 합니다. 같은 conversation에서 “방금 네가 만든 걸 채점해 봐”라고 하면 모델이 자기 기억으로 자기 결과물을 평가합니다. 이건 세션 격리가 깨진 상태입니다. 파일 수준에서는 분리됐지만 런타임에서는 같은 세션이라는 함정.

해결은 단순합니다. Evaluator를 별도 API 호출로 띄우되 Generator의 대화 히스토리를 컨텍스트에 넣지 않는 것. Evaluator는 오직 input.json(브리프)과 output.json(산출물)만 봅니다. Generator가 무슨 생각을 했는지 는 보지 않습니다. Evaluator 입장에서 output.json은 처음 보는 문서여야 합니다.

조건 3 · 다른 시스템 프롬프트

Generator의 시스템 프롬프트에는 “당신은 이런 스타일의 콘텐츠를 만드는 창작자입니다”가 들어갑니다. Evaluator의 시스템 프롬프트에는 “당신은 이런 콘텐츠의 품질을 1~5점으로 평가하는 심사자입니다. 창작자가 아닙니다”가 들어가야 합니다. 역할이 다르게 쓰여 있어야 같은 모델이라도 다른 인격으로 답합니다.

심사자 프롬프트에는 추가로 이런 문구가 유용합니다.

“이 산출물을 당신이 만들었다고 가정하지 마세요. 다른 사람이 만든 콘텐츠를 심사하는 입장입니다. 5점은 드뭅니다. 평균 3.0을 기준으로 채점하세요.”

마지막 한 줄이 자기 채점 관대함을 1점 정도 끌어내립니다.

5. 실전 코드 — generator.py + evaluator.py 최소 구조

이 책은 프레임워크를 팔지 않습니다. 다만 이 정도 골격이면 시작할 수 있습니다.

```
# generator.py
# 역할: 입력 브리프를 받아 산출물을 만든다. 평가는 여기서 하지 않는다.

from pathlib import Path
from some_llm_sdk import Client
import json

GEN_SYSTEM_PROMPT = Path("prompts/gen_system.md").read_text()

def generate(input_path: Path, output_path: Path) -> None:
    brief = json.loads(input_path.read_text())
    client = Client()
    response = client.messages.create(
        system=GEN_SYSTEM_PROMPT,
        messages=[{"role": "user", "content": json.dumps(brief)}],
        max_tokens=4000,
    )
    artifact = {
        "content": response.content,
        "brief_hash": brief["hash"],
        "generated_at": response.meta["timestamp"],
        # 중요: 자기 평가 점수를 여기 넣지 않는다.
    }
    output_path.write_text(json.dumps(artifact, ensure_ascii=False, indent=2))
```

```
# evaluator.py
# 역할: output.json을 입력으로 받아 점수와 판정을 verdict.json으로 낸다.
# Generator의 대화 히스토리나 내부 상태를 받지 않는다.

from pathlib import Path
from some_llm_sdk import Client
import json, yaml
```

```

EVAL_SYSTEM_PROMPT = Path("prompts/eval_system.md").read_text()
RUBRIC = yaml.safe_load(Path("rubric.yaml").read_text())

def evaluate(artifact_path: Path, verdict_path: Path) -> None:
    artifact = json.loads(artifact_path.read_text())
    client = Client()
    user_message = {
        "rubric": RUBRIC,
        "artifact": artifact["content"],
        # brief_hash만 참조용. 내용은 Evaluator가 독립 판단한다.
        "brief_hash": artifact["brief_hash"],
    }
    response = client.messages.create(
        system=EVAL_SYSTEM_PROMPT,
        messages=[{"role": "user", "content": json.dumps(user_message)}],
        max_tokens=800,
    )
    verdict = json.loads(response.content)
    verdict_path.write_text(json.dumps(verdict, ensure_ascii=False, indent=2))

```

두 파일의 총합이 60줄 남짓입니다. 하네스는 거대한 것이 아닙니다. 구조가 맞게 쪼개져 있느냐가 전부입니다.

6. 한 가지 함정 · “더 큰 모델이 Evaluator다” 는 착각

“Generator는 작은 모델에 맡기고, Evaluator는 큰 모델에 맡기자”라는 설계가 자주 등장합니다. 모델 체급이 다르면 분리가 된 것처럼 보이기 때문입니다. 이것은 부분적으로만 맞습니다.

체급이 다른 것은 도움이 됩니다. 큰 모델이 작은 모델의 결함을 더 잘 찾습니다. 그러나 체급이 분리의 본질은 아닙니다. 본질은 §4의 세 조건 — 다른 파일, 다른 세션, 다른 시스템 프롬프트. 두 Evaluator 설계를 비교하면 금방 보입니다.

- **A 설계:** 작은 모델 Generator + 큰 모델 Evaluator, 같은 세션에서 동시 호출
- **B 설계:** 같은 중급 모델 Generator + Evaluator, 단 다른 세션 + 다른 시스템 프롬프트 + 파일로 핸드오프

경험상 B 설계의 Evaluator가 A 설계의 Evaluator보다 일관되게 엄격합니다. 체급보다 컨텍스트 격리가 더 결정적입니다. 체급은 덧셈이지만 격리는 곱셈입니다.

7. 의심스러운 신호 · 당신의 파이프라인이 Gen/Eval 미분리인지 확인하는 5개 질문

- ☐ Generator의 호출 로그와 Evaluator의 호출 로그가 같은 파일 또는 같은 트레이스 ID에 기록된다.
- ☐ Evaluator가 “이 콘텐츠는 좋습니다”라고만 말하고 구체적 약점 한 줄도 없다.
- ☐ 재생성 루프가 3회를 넘는 경우가 한 주에 5회 이상 발생한다.
- ☐ 자기 점수 평균이 4.0 이상이고, 한 달 넘게 거의 변하지 않는다.
- ☐ 파이프라인 API 비용이 예산의 3배 이상으로 잘못 찍혀 당황한 적이 있다.

두 개 이상 체크되면 P1 점수 1~2점으로 보시면 됩니다.

8. 내일 30분에 할 것

- ☐ Evaluator가 Generator와 같은 세션·같은 파일·같은 시스템 프롬프트 인 파이프라인 하나를 고르세요.
- ☐ Evaluator를 다른 파일로 옮기세요. 그 하나만으로 3분이면 끝납니다.
- ☐ Evaluator의 시스템 프롬프트에 “이 콘텐츠를 당신이 만들었다고 가정하지 마세요. 5점은 드뭅니다”를 추가하세요.
- ☐ 다음 날 파이프라인을 한 번 돌리고, 이전 산출물의 자기 점수와 오늘 산출물의 외부 점수를 비교해 보세요. 간극이 1점 이상이라면 P1 교정이 의미 있었다는 증거입니다.

인용·참고

- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24 — P1 Generator-Evaluator Separation 원출처.
- Source: Anthropic Engineering, Effective harnesses for long-running agents — 선행 포스트의 평가자 분리 서술.

- Source: 저자의 운영 스튜디오 Quiet Ark의 2026-02 ~ 2026-04 파이프라인 개선 기록 (가상 도메인 DeckBrew News Curation / PageMark Blog · SNS · Card News / Quiet Ark Meme Generation 로 재구성).

Ch 4 · P3 + P5 — 검증 계약과 가치 인코딩

0. 이 장의 문장 하나

`validate.py` 가 없는 파이프라인은 파이프라인이 아닙니다. 그냥 세션입니다. 거기에 `rubric.yaml` 까지 더 해지면 비로소 품질이라는 것을 숫자로 말할 수 있게 됩니다. 이 장은 그 두 파일 한 쌍을 다룹니다.

1. 왜 P3와 P5를 같이 다루는가

책의 골격을 보면 Ch 3은 P1 하나를, Ch 5는 P4 하나를, Ch 6은 P2와 P6 쌍을 다룹니다. 이 장만 P3과 P5를 묶어 다룹니다. 이유는 두 원칙이 같은 질문을 다른 각도에서 답하기 때문입니다.

P3 (Sprint Contract)은 “최소 기준을 통과했느냐?” 를 묻습니다. 예/아니요. JSON 스키마가 맞는지, 길이가 범위 안에 있는지, 금치어가 없는지. 기계가 자동으로 답할 수 있는 질문들입니다. 통과하면 PASS, 못 하면 REJECT. 이진적입니다.

P5 (Value Encoding)는 “얼마나 좋은가?” 를 묻습니다. 1점에서 5점. structure 몇 점, tone 몇 점, hook 몇 점. 축 여러 개에 가중치를 곱해 평균을 낸 숫자가 품질 점수가 됩니다. 연속적입니다.

두 질문이 한 쌍인 이유는 최소 기준만 통과하면 되는 결과물도 없고, 최대 품질만 추구하는 결과물도 없기 때문입니다. PASS한 것 중에서 좋은 것을 골라야 하고, 좋다고 해도 최소 기준을 못 넘으면 공개 못 합니다. `validate`가 문이고 `rubric`이 저울입니다. 문을 먼저 지나야 저울에 올라갈 자격이 생깁니다.

2. `validate.py` — 네 가지 자동 검증

`validate`는 네 가지를 기계적으로 검사합니다. 이 네 가지가 파이프라인마다 약간의 변주가 있을 뿐, 큰 뼈대는 공통입니다.

2-1. JSON 스키마 검증

산출물이 약속된 필드를 전부 가지고 있는지 확인합니다. 필드 타입이 맞는지도요. 가장 기초적이지만 가장 자주 깨지는 부분입니다. LLM이 때때로 “brief_hash” 필드를 깜빡 잊거나, “quality_score”를 문자열 “4.5”로 반환하기 때문입니다.

```
# validate.py 일부
import json
from jsonschema import validate as jsonschema_validate, ValidationError

SCHEMA = {
    "type": "object",
    "required": ["content", "brief_hash", "generated_at"],
    "properties": {
        "content": {"type": "string", "minLength": 1},
        "brief_hash": {"type": "string", "pattern": "^[a-f0-9]{8,}$"},
        "generated_at": {"type": "string", "format": "date-time"},
    },
}

def check_schema(artifact: dict) -> list[str]:
    try:
        jsonschema_validate(artifact, SCHEMA)
        return []
    except ValidationError as e:
        return [f"schema: {e.message}"]
```

2-2. 길이 범위 검사

콘텐츠 길이가 약속된 상하한 안에 있는가. LLM이 짧게 잘라 버리거나, 반대로 하염없이 늘어뜨리는 두 실패를 잡아 냅니다.

```
def check_length(content: str, min_chars=300, max_chars=4000) -> list[str]:
    n = len(content)
    if n < min_chars:
        return [f"length: {n} < {min_chars}"]
    if n > max_chars:
        return [f"length: {n} > {max_chars}"]
    return []
```

2-3. 금치어 필터

브랜드 가이드 또는 법무적 제약에 맞지 않는 표현이 들어갔는지 검사합니다. 서비스마다 금치어 목록이 다르고, 목록이 유통기한도 있습니다. 이 검사가 있으면 브랜드가 조용히 무너지는 일을 막습니다.

```
BANNED_WORDS = ["무조건", "완벽보장", "절대안전", "반드시성공"]
```

```
def check_banned(content: str) -> list[str]:  
    errors = []  
    for w in BANNED_WORDS:  
        if w in content:  
            errors.append(f"banned: '{w}' present")  
    return errors
```

2-4. 품질 점수 하한

rubric으로 계산된 가중 평균이 최소 기준 이상인지 확인합니다. P5 영역과 이 지점에서 만납니다. validate가 단순히 "PASS/REJECT"만 내는 것이 아니라, 품질 하한까지 함께 거절합니다. rubric 적용 후 가중 평균 2.5 미만이면 REJECT.

```
def check_quality(rubric_scores: dict, threshold=2.5) -> list[str]:  
    total = sum(s * rubric_scores["weights"][k]  
                for k, s in rubric_scores["scores"].items())  
    if total < threshold:  
        return [f"quality: {total:.2f} < {threshold}"]  
    return []
```

2-5. 네 가지를 합친 contract 함수

```
def validate_contract(artifact: dict, rubric_scores: dict) -> dict:  
    errors = []  
    errors += check_schema(artifact)  
    errors += check_length(artifact["content"])  
    errors += check_banned(artifact["content"])  
    errors += check_quality(rubric_scores)  
    return {  
        "verdict": "REJECT" if errors else "PASS",  
        "errors": errors,  
    }
```

전체 validate.py가 80줄이면 끝납니다. 30분 작업입니다. 30분 하지 않으면 대신 조용한 품질 하락 2주를 겪습니다.

2-6. 실제 돌아가는 validate.py 전체

참고용으로 실제 한 뭉치 파일 전체를 수록합니다. 80줄입니다. 이 상태로 복사해 자기 파이프라인의

`validate.py` 초안으로 쓸 수 있습니다.

```

# validate.py - P3 Sprint Contract 실전 구현
# 사용: python validate.py <output.json> <verdict.json>
# 의존: pip install jsonschema pyyaml

import json
import sys
from pathlib import Path
from jsonschema import validate as jsonschema_validate, ValidationError
import yaml

# ----- 1. 스키마 정의 (파이프라인마다 교체) -----

ARTIFACT_SCHEMA = {
    "type": "object",
    "required": ["content", "brief_hash", "generated_at"],
    "properties": {
        "content": {"type": "string", "minLength": 1},
        "brief_hash": {"type": "string", "pattern": "^[a-f0-9]{8,}$"},
        "generated_at": {"type": "string", "format": "date-time"},
        "generator_model": {"type": "string"},
    },
    "additionalProperties": True,
}

LENGTH_MIN = 300
LENGTH_MAX = 4000
QUALITY_MIN = 2.5
BANNED_WORDS = ["무조건", "완벽보장", "절대안전", "반드시성공"]

# ----- 2. 개별 체크 -----

def check_schema(artifact: dict) -> list[str]:
    try:
        jsonschema_validate(artifact, ARTIFACT_SCHEMA)
        return []
    except ValidationError as e:
        return [f"schema: {e.message}"]

def check_length(content: str) -> list[str]:
    n = len(content)
    if n < LENGTH_MIN:
        return [f"length: {n} < {LENGTH_MIN}"]
    if n > LENGTH_MAX:
        return [f"length: {n} > {LENGTH_MAX}"]
    return []

def check_banned(content: str) -> list[str]:
    return [f"banned: '{w}'" for w in BANNED_WORDS if w in content]

def check_quality(rubric_scores: dict) -> list[str]:
    scores = rubric_scores.get("scores", {})

```

```

weights = rubric_scores.get("weights", {})
if not scores or not weights:
    return ["quality: missing scores or weights"]
total = sum(scores[k] * weights[k] for k in scores if k in weights)
if total < QUALITY_MIN:
    return [f"quality: {total:.2f} < {QUALITY_MIN}"]
return []

# ----- 3. 합친 contract -----

def validate_contract(artifact_path: Path, verdict_path: Path) -> dict:
    artifact = json.loads(Path(artifact_path).read_text())
    verdict_existing = {}
    if Path(verdict_path).exists():
        verdict_existing = json.loads(Path(verdict_path).read_text())
    rubric_scores = verdict_existing.get("rubric_scores", {})

    errors = []
    errors += check_schema(artifact)
    errors += check_length(artifact.get("content", ""))
    errors += check_banned(artifact.get("content", ""))
    errors += check_quality(rubric_scores)

    verdict = {
        **verdict_existing,
        "contract_errors": errors,
        "verdict": "REJECT" if errors else "PASS",
    }
    Path(verdict_path).write_text(json.dumps(verdict, ensure_ascii=False, indent=2))
    return verdict

if __name__ == "__main__":
    out = validate_contract(Path(sys.argv[1]), Path(sys.argv[2]))
    print(json.dumps(out, ensure_ascii=False, indent=2))

```

QUALITY_MIN · LENGTH_MIN · BANNED_WORDS 만 파이프라인에 맞춰 바꾸면 80% 재사용. 스키마 정의는 ARTIFACT_SCHEMA 한 블록을 도메인에 맞게 교체.

3. validate가 잡히는 것과 잡히지 않는 것의 경계선

validate가 잡음	validate가 못 잡음
JSON 필드 누락	주장과 근거 배치의 설득력

validate가 잡음	validate가 못 잡음
길이 상·하한 위반	인용 선택의 자연스러움
금칙어 등장	문단 사이 흐름
스키마 타입 오류	톤의 미묘한 일관성
brief_hash 불일치	독자가 느낄 신선함

왼쪽은 기계가 검사 가능한 영역, 오른쪽은 가치 판단이 필요한 영역입니다. 오른쪽이 P5의 영역입니다.

4. rubric.yaml — 가치를 축으로 쪼개는 법

“좋은 콘텐츠”를 루브릭으로 옮기는 작업이 모든 P5 도입에서 가장 오래 걸리는 지점입니다. 단순히 “좋은”을 쪼개는 것이 아니라, 다섯 또는 일곱 개의 측정 가능한 축으로 쪼개는 것입니다.

4-1. 축 고르는 원칙 — 서로 독립적이어야 함

두 축이 같은 것을 측정하면 가중치 설계가 의미를 잃습니다. “structure”와 “flow”를 같이 쓰면 한 축이 오를 때 다른 축도 같이 오릅니다. 이런 축들은 한 축으로 합쳐야 합니다. 좋은 축 후보 일반 목록.

- **structure** — 주장·근거·예시·결론의 층위가 분리되어 있는가
- **evidence** — 주장마다 근거(직인용·수치·구체 사례)가 붙는가
- **tone** — 브랜드 톤과 일관된 말투인가
- **hook** — 첫 3초 안에 시선이 멎는가
- **uniqueness** — 저자 특유의 관점·조어·비유가 등장하는가
- **actionability** — 독자가 덮은 직후 무엇을 할지 아는가
- **length calibration** — 의도된 길이 범위 안에서 밀도가 맞는가

각 축이 정확히 어떤 현상을 측정하는지를 정의해 줘야 합니다. 추상적인 이름만으로는 평가자가 주관으로 돌아갑니다.

4-2. 1·3·5점 기준 — 구체적 장면으로 정의

각 축에 1점, 3점, 5점이 구체적으로 무엇인지 써 둡니다. 2점과 4점은 내삽. 이 정의가 없으면 평가자가 “적당히” 3점을 주게 됩니다. 모든 점수가 3점 근처에 몰리면 루브릭이 무의미해집니다.

DeckBrew Newsletter 파이프라인의 실제 rubric.yaml 7축 발췌.

```
# rubric.yaml · DeckBrew Newsletter Pipeline
axes:
  - name: structure
    weight: 0.15
    scale:
      1: "주장·근거·예시 구분 없이 한 덩어리"
      3: "구분은 있으나 하나가 과도하게 짧거나 길다"
      5: "3요소 균형 + 결론까지 명시적으로 분리"
  - name: evidence
    weight: 0.20
    scale:
      1: "모든 주장이 근거 없음"
      3: "일부 주장에만 근거"
      5: "모든 주장에 직인용·수치·케이스 중 하나 이상"
  - name: tone
    weight: 0.15
    scale:
      1: "브랜드 톤에서 이탈한 구어체·외국어 혼재"
      3: "대부분 톤 유지, 한두 문장에서 이탈"
      5: "전체 문서가 브랜드 보이스 가이드 준수"
  - name: hook
    weight: 0.15
    scale:
      1: "배경 설명으로 시작, 첫 문단 지나도 핵심 안 나옴"
      3: "두 번째 문단에서 핵심 등장"
      5: "첫 문장부터 독자 상황 직격"
  - name: uniqueness
    weight: 0.15
    scale:
      1: "일반론 나열, 이 뉴스레터만의 관점 없음"
      3: "저자 고유 관점 한 군데 등장"
      5: "여러 문단에 저자 고유 비유·조어 등장"
  - name: actionability
    weight: 0.10
    scale:
      1: "독자가 읽고 난 뒤 무엇을 할지 모호"
      3: "다음 행동이 암시되나 구체적이지 않음"
      5: "다음 행동이 명시적으로 제시, 소요 시간까지"
  - name: length_calibration
    weight: 0.10
    scale:
      1: "의도 길이의 50% 미만 또는 200% 초과"
      3: "의도 범위 안이나 밀도가 고르지 않음"
      5: "의도 범위 안 + 문단별 밀도 균일"
```

```
min_total: 2.8 # 가중 평균이 이 값 미만이면 REJECT
```

가중치 합은 1.00이어야 합니다. 검사용 스크립트 한 줄.

```
assert abs(sum(a["weight"] for a in rubric["axes"]) - 1.0) < 1e-6
```

4-2-b. rubric.yaml 전체 7축 (실측 풀 예시)

위의 §4-2에서 3축만 발췌했습니다. 이번에는 DeckBrew Newsletter의 실제 rubric.yaml 7축 전체를 수록합니다. 파이프라인마다 복사해 축 이름과 scale 장면만 도메인에 맞게 바꾸면 바로 쓸 수 있습니다.

```
# rubric.yaml · DeckBrew Newsletter Pipeline · 2026-04-17 버전
version: 1
axes:
- name: structure
  weight: 0.15
  scale:
    1: "주장·근거·예시 구분 없이 한 덩어리"
    3: "구분은 있으나 하나가 과도하게 짧거나 길다"
    5: "3요소 균형 + 결론까지 명시적으로 분리"

- name: evidence
  weight: 0.20
  scale:
    1: "모든 주장이 근거 없음"
    3: "일부 주장에만 근거"
    5: "모든 주장에 직인용·수치·케이스 중 하나 이상"

- name: tone
  weight: 0.15
  scale:
    1: "브랜드 톤에서 이탈한 구어체·외국어 혼재"
    3: "대부분 톤 유지, 한두 문장에서 이탈"
    5: "전체 문서가 브랜드 보이스 가이드 준수"

- name: hook
  weight: 0.15
  scale:
    1: "배경 설명으로 시작, 첫 문단 지나도 핵심 안 나옴"
    3: "두 번째 문단에서 핵심 등장"
    5: "첫 문장부터 독자 상황 직격"

- name: uniqueness
  weight: 0.15
  scale:
    1: "일반론 나열, 이 뉴스레터만의 관점 없음"
    3: "저자 고유 관점 한 군데 등장"
    5: "여러 문단에 저자 고유 비유·조어 등장"
```

```

- name: actionability
  weight: 0.10
  scale:
    1: "독자가 읽고 난 뒤 무엇을 할지 모호"
    3: "다음 행동이 암시되나 구체적이지 않음"
    5: "다음 행동이 명시적으로 제시, 소요 시간까지"

- name: length_calibration
  weight: 0.10
  scale:
    1: "의도 길이의 50% 미만 또는 200% 초과"
    3: "의도 범위 안이나 밀도가 고르지 않음"
    5: "의도 범위 안 + 문단별 밀도 균일"

min_total: 2.8
notes: |
  - min_total 미만이면 validate.py가 REJECT
  - 평가자 프롬프트에는 '5점은 드뭅니다. 평균 3.0 기준' 문구 포함 필수
  - 가중치 합 검증은 validate.py 실행 시 1회 수행 (assert)

```

가중치 합 검증을 위해 이 파일을 로드하는 쪽에서 한 줄 추가.

```

import yaml, pathlib
rubric = yaml.safe_load(pathlib.Path("rubric.yaml").read_text())
assert abs(sum(a["weight"] for a in rubric["axes"]) - 1.0) < 1e-6, \
    "rubric weight sum is not 1.0"

```

이 한 줄이 8장 실패 4번의 가중치 합 1.2 6주 방치 사고를 막습니다.

4-3. 축 개수의 Sweet Spot

경험상 5~7축이 가장 좋습니다. 3축 이하는 해상도가 너무 낮고, 9축 이상은 각 축의 가중치가 0.10 미만으로 떨어져 한 축 한 축이 가중합에 미치는 영향이 미미해집니다. 평가자가 “이 축은 1점이든 5점이든 상관없잖아”라고 느끼기 시작하면 루브릭이 형해화됩니다.

5. 평가자 프롬프트의 두 문장

rubric.yaml이 있어도 평가자에게 어떻게 쓰라고 명시하지 않으면 평가자는 느슨해집니다. 평가자 시스템 프롬프트에 반드시 포함할 두 문장.

“5점은 드뭅니다. 평균 3.0을 기준으로 채점하세요.”

“각 축에 점수를 준 근거 한 줄을 함께 출력하세요.”

첫 문장은 점수 팽창을 억제합니다. 두 번째 문장은 근거 요구가 곧 채점 품질이기 때문입니다. 근거를 못 쓰면 점수도 못 줍니다. 이 한 문장이 “왜 5점인지 모르는 5점”을 사라지게 합니다.

6. validate + rubric 실전 흐름

두 파일이 파이프라인 안에서 어떻게 돌아가는지의 실제 흐름.

1. generator.py 가 output.json 생성
2. evaluator.py 가 rubric.yaml 을 적용, rubric_scores 를 verdict.json 에 기록
3. validate.py 가 output.json + rubric_scores 를 받아 contract 검사
 - └ JSON 스키마 체크
 - └ 길이 체크
 - └ 금칙어 체크
 - └ 품질 하한 체크
4. verdict.json 의 최종 verdict:
 - PASS → next_agent_input.json 으로 핸드오프
 - REJECT → errors 배열과 함께 regen_request.json 으로 Generator 재호출

재호출 시 중요한 점. Generator에게 전달하는 regen_request.json에는 errors 배열과 rubric에서 3점 이하로 찍힌 축 목록이 들어갑니다. 총점을 알려 주지 않습니다. 총점을 알려 주면 Generator가 그 숫자를 맞추려 하면서 축간 균형이 깨집니다. 약점만 알려 주는 편이 낫습니다.

7. 도입 전후 실측

2026-02 시점에 네 개의 파이프라인에 validate + rubric을 함께 도입하고 2026-04 시점까지 두 달간 관찰한 기록.

파이프라인	P3 전	P3 후	P5 전	P5 후	가중합 전	가중합 후
PageMark Blog Generation	2	6	3	7	3.05	4.10
DeckBrew Newsletter	2	6	3	7	3.50	4.85
DeckBrew Card News Image	2	5	2	7	3.25	4.40
Nightwatch Mentoring Review	2	6	3	7	3.35	5.30

P3 점수가 2에서 6으로 일관되게 오른 이유: validate.py 도입 자체가 P3의 실체이기 때문입니다. 없었다가 생기는 것이니 0점대에서 시작해 6점대에 안착합니다. P5도 비슷한 점프. 다만 7장에서 얘기했듯 DeckBrew Card News Image 는 루브릭 도입 중반에 점수가 일시적으로 내려갔다가 복구됐습니다. 이 사이클이 정상입니다.

도입으로 바뀌는 진짜 것은 가중합 숫자가 아닙니다. 리뷰 회의의 시간이 바뀝니다. rubric 도입 전 리뷰 한 건 평균 28분에서 도입 후 11분으로 줄었습니다. “좋아 보여요” vs “글썸요”의 추상 논쟁이 “hook 4점, structure 3점, evidence 2점”의 구체 논쟁으로 옮겨갔기 때문입니다. 구체 논쟁이 빠른 논쟁입니다.

8. 의심스러운 신호 · rubric은 있는데 안 돌아가는 파이프라인

- ☐ rubric.yaml은 존재하는데 지난 30일간 수정된 기록이 없다.
- ☐ 루브릭의 1·3·5점 정의가 추상 형용사(“더 자연스러운”, “더 매끄러운”)로만 쓰여 있다.
- ☐ 가중 평균이 항상 3.5에서 4.5 사이. 1점과 5점이 거의 안 나온다.
- ☐ 평가자가 점수만 내고 각 축 근거 한 줄을 안 쓴다.
- ☐ validate.py가 JSON 스키마 체크만 한다. 길이·금칙어·품질 하한 중 둘 이상이 없다.

두 개 이상이면 P5 점수가 허수일 가능성이 큼니다. 2장의 표를 다시 매겨야 할 수 있습니다.

9. 내일 30분에 할 것

- ☐ 파이프라인 하나를 고르세요. 이미 돌아가고 있지만 validate/rubric이 없는 것.

- ☐ validate.py 파일을 만들어 네 함수(schema/length/banned/quality) 중 적어도 두 개를 구현하세요. 30 줄이면 됩니다.
 - ☐ rubric.yaml 파일을 만들어 3개 축만 1·3·5점 구체 장면으로 정의하세요. structure / evidence / tone 만으로 시작.
 - ☐ 가중치 합이 1.0이 되도록 조정(예: 0.4 / 0.4 / 0.2).
 - ☐ 최근 3건 산출물에 rubric을 적용해 점수 매김. 세 건이 전부 3.5점 이하면 원고가 잘못된 게 아니라 파이프라인 설계가 잘못된 것입니다.
-

인용·참고

- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24 — P3 Sprint Contract / P5 Grading Criteria Value Encoding.
- Source: 저자의 운영 스튜디오 Quiet Ark의 2026-02 ~ 2026-04 사이클 (DeckBrew Newsletter / PageMark Blog / Nightwatch Mentoring Review 실측 가상 재구성).

Ch 5 · P4 파일 기반 핸드오프 — 파일로 말하세요

0. 이 장의 문장 하나

대화는 휘발합니다. 파일은 남습니다. 에이전트끼리 대화로 일하는 파이프라인은 이름만 파이프라인이지 실체는 세션입니다. 세션은 죽으면 끝납니다. 파이프라인은 죽어도 재개됩니다. 그 차이가 P4 하나에서 갈립니다.

1. 파이프라인 vs 세션 — 단 한 번의 종료가 만드는 차이

두 개의 동일한 작업을 상상해 봅시다. 12시간짜리 멘토링 리뷰 작업. 멘티 80명의 제출물을 LLM이 하나씩 읽고 피드백을 붙입니다.

A 설계 — 대화 기반. 한 Claude 세션을 열고, 80건을 순서대로 던집니다. 각 건의 피드백은 세션 히스토리 안에 쌓입니다. 마지막에 전체를 한 번에 내보냅니다. 6시간 지점에서 세션이 OOM으로 죽었습니다. 히스토리가 통째로 날아갔습니다. 처음부터 다시.

B 설계 — 파일 기반. 각 건의 입력을 `input_{id}.json` 으로 저장하고, 처리 결과를 `output_{id}.json` 으로 씁니다. 6시간 지점에서 죽어도, 다음 실행은 `output_` 파일이 이미 있는 ID를 건너뛵니다. 6시간 추가로 돌면 완성. 총 12시간 + 세션 전환 오버헤드 약 30분.

같은 작업, 같은 모델, 같은 피드백 품질. 죽음 한 번이 있느냐 없느냐에서 총 시간이 2배 차이가 납니다. 게다가 B 설계는 재개 가능이기 때문에 더 자주 죽어도 됩니다. 5시간마다 의도적으로 세션을 끊고 새 세션을 시작하면 컨텍스트 오염(6장의 P2)을 예방할 수 있습니다.

2. 핸드오프의 기본 구조

```
input.json
↓ generator.py
output.json
```

```

↓ evaluator.py
verdict.json
↓ validate.py
├─ PASS → next_agent_input.json
└─ REJECT → regen_request.json & generator.py

```

네 개의 파일이 에이전트 네 명 사이를 오갑니다. 각 파일이 해당 에이전트의 단 하나의 입력이어야 합니다. 이 것이 깨지면 핸드오프가 아니라 암묵 전달입니다.

2-1. 각 파일의 최소 스키마

```

// input.json
{
  "brief_hash": "a1b2c3d4",
  "brief": {
    "topic": "...",
    "audience": "...",
    "constraints": ["..."]
  },
  "created_at": "2026-04-17T03:17:00Z"
}

```

```

// output.json
{
  "brief_hash": "a1b2c3d4",
  "content": "...",
  "generated_at": "2026-04-17T03:19:22Z",
  "generator_model": "<가상 모델명>",
  "generator_commit": "<가상 해시>"
}

```

```

// verdict.json
{
  "brief_hash": "a1b2c3d4",
  "rubric_scores": {
    "structure": 4,
    "evidence": 3,
    "tone": 5,
    "hook": 4,
    "uniqueness": 3,
    "actionability": 4,
    "length_calibration": 4
  },
  "weighted_total": 3.85,
  "contract_errors": [],
  "verdict": "PASS",
  "evaluator_model": "<가상 모델명>",
}

```

```
"evaluated_at": "2026-04-17T03:20:11Z"
}
```

`brief_hash` 가 네 파일 모두에 등장합니다. 이 해시가 한 건의 작업을 엮는 단 하나의 연결 고리입니다. 해시로 엮이지 않은 파일은 파이프라인의 일부가 아닙니다.

2-2. 파일 이름의 원칙

- 파일명에 timestamp + `brief_hash`가 들어갈 것
- 에이전트 이름이 파일명에 들어가면 안 됨 (에이전트가 바뀌어도 파일 의미는 남아야 함)
- 확장자는 `.json` 단일 (에이전트 간 통신용으로는 `yaml·markdown` 혼용 금지)
- 디렉토리는 단계별로 분리: `runs/{hash}/01_input.json`, `02_output.json`, `03_verdict.json`, `04_next.json`

2-3. 파이프라인 전체 디렉토리 구조 (실전)

한 파이프라인의 전체 레이아웃. 신규 파이프라인을 시작할 때 이 구조를 그대로 복사해 쓸 수 있습니다.

```
pipelines/<pipeline_name>/
├── CLAUDE.md                # 이 파이프라인의 규칙 · 도메인 언어 · 금지 행동
├── generator.py             # P1: 생성자. evaluator.py를 import 금지
├── evaluator.py             # P1: 평가자. generator 히스토리 미참조
├── validate.py              # P3: 자동 검증 계약 (Ch4 §2-6)
├── rubric.yaml              # P5: 가치 축 7개 + 가중치 (Ch4 §4-2-b)
├── prompts/
│   ├── gen_system.md       # Generator용 시스템 프롬프트
│   └── eval_system.md      # Evaluator용 시스템 프롬프트
├── schemas/
│   ├── input.schema.json   # 브리프 스키마
│   ├── output.schema.json  # 산출물 스키마
│   └── verdict.schema.json  # 판정 스키마
├── runs/
│   ├── 2026-04-24_a1b2c3d4/ # 한 건당 하나의 디렉토리
│   │   ├── 01_input.json
│   │   ├── 02_output.json
│   │   ├── 03_verdict.json
│   │   └── 04_next.json     # PASS 시에만 생성
│   └── 2026-04-24_e5f6a7b8/
│       ├── 01_input.json
│       ├── 02_output.json
│       ├── 03_verdict.json
│       └── 99_regen_request.json # REJECT 시 재생성 요청
├── archive/                 # 30일 지난 runs 이동
└── README.md                # 운영자 개요 · 티어 · HOC 사이클 기록
```

한 파이프라인이 하나의 디렉토리로 완결됩니다. 운영자 교체 시 이 디렉토리만 넘겨 받으면 맥락이 이전됩니다. P4 점수를 6점 이상으로 올리려면 이 구조가 기본입니다.

2-4. 세 가지 JSON 스키마 전체

참고용으로 세 스키마 파일의 실제 내용. `schemas/` 아래에 그대로 넣으면 됩니다.

`schemas/input.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Pipeline Input",
  "type": "object",
  "required": ["brief_hash", "brief", "created_at"],
  "properties": {
    "brief_hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{8,}$",
      "description": "브리프 해시. 한 건을 관통하는 유일 식별자"
    },
    "brief": {
      "type": "object",
      "required": ["topic"],
      "additionalProperties": true
    },
    "created_at": {
      "type": "string",
      "format": "date-time"
    }
  },
  "additionalProperties": false
}
```

`schemas/output.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Pipeline Output (Generator Artifact)",
  "type": "object",
  "required": ["brief_hash", "content", "generated_at"],
  "properties": {
    "brief_hash": {"type": "string", "pattern": "^[a-f0-9]{8,}$"},
    "content": {"type": "string", "minLength": 1},
    "generated_at": {"type": "string", "format": "date-time"},
    "generator_model": {"type": "string"},
    "generator_commit": {"type": "string"}
  },
}
```

```
"additionalProperties": false
}
```

schemas/verdict.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Pipeline Verdict (Evaluator + Validator)",
  "type": "object",
  "required": ["brief_hash", "verdict", "rubric_scores"],
  "properties": {
    "brief_hash": {"type": "string", "pattern": "^[a-f0-9]{8,}$"},
    "verdict": {"enum": ["PASS", "REJECT"]},
    "rubric_scores": {
      "type": "object",
      "required": ["scores", "weights", "weighted_total"],
      "properties": {
        "scores": {
          "type": "object",
          "patternProperties": {"^[a-z_]+$": {"type": "number", "minimum": 0, "maximum": 5}}
        },
        "weights": {
          "type": "object",
          "patternProperties": {"^[a-z_]+$": {"type": "number", "minimum": 0, "maximum": 1}}
        },
        "weighted_total": {"type": "number", "minimum": 0, "maximum": 5}
      }
    },
    "contract_errors": {
      "type": "array",
      "items": {"type": "string"}
    },
    "evaluator_model": {"type": "string"},
    "evaluated_at": {"type": "string", "format": "date-time"}
  },
  "additionalProperties": false
}
```

세 스키마가 한 파이프라인의 단 하나의 계약입니다. 에이전트를 바꾸든 모델을 바꾸든 이 세 스키마를 깨지 않는 한 파이프라인은 작동합니다.

3. 대화가 아니라 파일이어야 하는 5가지 이유

3-1. 복구 가능성

죽어도 마지막 파일부터 재개. 이미 §1에서 다뤘습니다.

3-2. 인터페이스 안정성

에이전트를 교체해도 파일 포맷이 같으면 나머지 파이프라인은 그대로. Generator를 작은 모델로 바꾸고 싶을 때 output.json 스키마만 지키면 됩니다. 대화 기반이면 대화 컨텍스트까지 같이 바뀌어야 합니다.

3-3. 디버깅 가능성

실패한 건의 input/output/verdict 세 파일만 있으면 재현이 가능합니다. “왜 이 건이 REJECT 됐는지”를 파일 세 개로 설명할 수 있습니다. 대화 기반은 전체 세션 히스토리를 다시 꺼내야 합니다.

3-4. 병렬화 가능성

파일 기반은 같은 디렉토리에 여러 Generator를 동시에 띄울 수 있습니다. 각자 자기 hash의 파일만 건드리면 충돌 안 납니다. 대화 기반은 세션이 공유 리소스라 병렬화가 어렵습니다.

3-5. 감사 가능성

3개월 뒤 “이 파이프라인 왜 품질이 이래?” 질문이 나오면, `runs/` 디렉토리 몇 개만 열어 봐도 답이 나옵니다. 대화 기반은 로그가 있다 해도 특정 세션의 히스토리를 복원하는 게 어렵습니다.

4. 파일로 적히지 않은 통신 = 파이프라인의 구멍

이 챕터의 가장 중요한 검사 항목은 “파일로 적히지 않은 통신이 몇 곳이나 있는가?” 입니다. 흔한 구멍들.

4-1. 공유 전역 변수

Python 모듈 레벨의 dict 하나에 상태를 쌓아 놓고 여러 에이전트가 참조. 코드상 깔끔해 보이지만 프로세스가 죽으면 전역 변수도 같이 죽습니다. 복구 불가.

4-2. 메모리 캐시

Redis나 memcached에 중간 결과를 넣고 다음 에이전트가 꺼내 씀. Redis가 파일 역할을 하니까 P4 관점에서 OK처럼 보이지만, 캐시는 TTL이 있어 영속적이지 않습니다. 1시간 뒤에 없어집니다. 감사·재현에는 쓸 수 없습니다. 원본은 반드시 디스크 파일로 따로 남겨야 합니다.

4-3. LLM 세션 히스토리

“방금 내가 만든 걸 평가해 줘” 라는 요청이 세션 안에서 일어나고, 평가 결과가 세션에만 존재합니다. 복구도, 디버깅도, 감사도 불가능합니다. 파일로 적히지 않은 통신 중 가장 치명적입니다.

4-4. stderr

에이전트가 경고 메시지를 stderr로만 출력하고 파일로 찍지 않음. 파이프라인이 실패했을 때 왜 실패했는지를 알려 주는 단서가 휘발합니다. stderr은 동시에 파일로도 남겨야 합니다.

4-5. 사람의 머릿속

“이 파이프라인은 이럴 때 이렇게 처리한다”는 규칙이 운영자의 머릿속에만 있음. 운영자가 바뀌면 규칙도 사라집니다. 규칙은 `rules.md` 같은 파일에 적혀 있어야 합니다.

다섯 구멍 중 하나라도 있으면 P4 점수가 5점 이상 나올 수 없습니다.

5. 파일 기반 파이프라인 실측 — Nightwatch Mentoring Review

7장의 표에서 9번 Nightwatch Mentoring Review의 P4 점수는 5점입니다. 특별히 높은 편이 아닙니다. 이유는 단 하나의 구멍 — 실패 시 운영자 판단이 머릿속에 남아 있음. 다른 네 구멍은 다 막혀 있는데, 이 마지막 하나가 P4를 5점으로 묶어 두고 있습니다.

통신 경로	상태
멘티 제출물 → Generator	<code>runs/{id}/01_submission.json</code> ✓
Generator → Evaluator	<code>runs/{id}/02_draft_review.json</code> ✓
Evaluator → 멘티 통지	<code>runs/{id}/03_final_review.json</code> ✓
Evaluator → 운영자 이슈 보고	<code>runs/{id}/04_issue_report.json</code> ✓
이슈 처리 규칙	운영자 머릿속 ✗

마지막 한 줄이 P4를 6점으로 올리지 못하게 합니다. 해결은 [docs/issue_handling_rules.md](#) 파일 하나. 30분 작업. 다음 분기에 이 파일을 추가하면 P4가 6점으로 올라가고, 가중합이 0.15 늘어납니다. 5.30에서 5.45로.

숫자로는 작아 보이지만 운영 관점에서 운영자 교체 가능성이 열린다는 점에서 큰 차이입니다. 문서화된 규칙 없이는 이 파이프라인이 운영자 개인의 소유물입니다.

6. 의심스러운 신호 · 파일 기반이 아닌 파이프라인의 증상

- ☐ 파이프라인이 죽었을 때 어디서 죽었는지 알려면 세션 로그를 뒤져야 한다.
- ☐ 어제 돌린 파이프라인의 중간 결과를 오늘 다시 꺼내 볼 수 없다.
- ☐ 파이프라인 재시작 = 항상 처음부터.
- ☐ 에이전트를 바꾸려면 다른 에이전트 코드도 같이 고쳐야 한다.
- ☐ 운영자만 아는 규칙이 세 개 이상 있다.

세 개 이상이면 P4 점수 1~2점입니다.

7. 파일 핸드오프 도입 로드맵

하루에 다 할 필요 없습니다. 4주 분할 권장.

Week 1 — 스키마 정의

`input.json`, `output.json`, `verdict.json` 세 개의 JSON 스키마를 파일로 써 둡니다. 전체 스키마를 쓸 필요는 없고 필수 필드 5~7개만. `brief_hash`, `created_at`, 도메인 특화 필드 두세 개.

Week 2 — 쓰기 단계 구현

Generator에서 `output.json`을 디스크에 쓰는 코드를 추가. Evaluator에서 `output.json`을 읽는 코드를 추가. 두 에이전트 사이의 메모리 전달을 파일로 교체.

Week 3 — 재개 가능성 구현

pipeline runner에서 이미 `output`이 존재하는 `brief_hash`는 건너뛰는 로직 추가. 이것 하나로 §1의 “죽음 한 번에 2배 시간” 문제가 해결됩니다.

Week 4 — 디렉토리 구조 정리

`runs/{hash}/01_input.json`, `02_output.json`, `03_verdict.json`, `04_next.json` 네 파일을 일관되게 저장. 로그 로테이션 스크립트 추가 (30일 이상 된 runs는 archive/로 이동).

8. 내일 30분에 할 것

- ☐ 파이프라인 하나를 골라 에이전트 간 통신 경로를 종이에 그리세요. 화살표가 몇 개일까요.
- ☐ 각 화살표 위에 파일 경로를 쓰세요. 파일이 아닌 것(전역 변수, LLM 세션, 머릿속)은 표시하세요.

- ☐ 파일이 아닌 화살표 중 하나를 고르세요.
 - ☐ 그 하나를 오늘 파일로 뽑으세요. 스키마는 필드 3개면 충분합니다.
 - ☐ 내일 파이프라인을 돌리면서 그 파일이 실제로 생성되는지 확인하세요.
-

인용·참고

- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24 — P4 File-Based Handoff 원출처.
- Source: A. Karpathy, LLM Wiki Concept, 2026-04 — 파일이 영속 지식의 최소 단위라는 관점.
- Source: 저자의 운영 스튜디오 Quiet Ark의 Nightwatch Mentoring Review 파이프라인 실측.

Ch 6 · P2 + P6 — 컨텍스트 불안과 노후화 감사

0. 이 장의 문장 하나

컨텍스트는 많다고 좋은 게 아니고, 오래됐다고 유지되는 것도 아닙니다. 이 두 오해가 파이프라인의 품질을 가장 조용히 깎아내립니다. P2와 P6이 쌍으로 묶이는 이유입니다.

1. 왜 P2와 P6이 쌍인가

3장에서 P1을 단독으로, 4장에서 P3와 P5를 쌍으로, 5장에서 P4를 단독으로 다뤘습니다. 이 장만 P2와 P6을 한꺼번에 다룹니다.

P2 (Context Anxiety Mitigation)는 컨텍스트가 너무 많아서 생기는 문제를 다룹니다. “맥락을 충분히 안 뵈나?” 라는 심리적 불안이 토큰을 계속 붓게 만들고, 그 토큰이 품질을 떨어뜨립니다.

P6 (Harness Staleness Audit)은 컨텍스트가 너무 오래돼서 생기는 문제를 다룹니다. 3개월 전 쓴 프롬프트에 단종된 모델명이 남아 있고, 예시로 넣은 URL은 죽었습니다.

두 원칙 모두 “컨텍스트의 건강”에 관한 문제입니다. 한쪽은 양의 문제이고, 다른 쪽은 연식의 문제입니다. 실전에서는 두 문제가 자주 같이 발생합니다. 오래된 파이프라인은 컨텍스트가 누적된 채로 늙어가기 때문입니다. 그래서 쌍으로 다룹니다.

2. P2 · 컨텍스트 불안

“모델에 맥락을 충분히 뵈나?” 이 문장에서 불안이 시작됩니다. 불안의 해소 방법이 한 가지 — 더 넣는 것. 그래서 사람들은 컨텍스트에 계속 토큰을 붓습니다. “혹시 몰라서” 과거 산출물도 넣고, “참고용으로” 브랜드 가이드 전문도 넣고, “도움이 될까 봐” 경쟁사 사례도 넣습니다. 8,000 토큰. 10,000 토큰. 12,000 토큰.

품질은 같이 올라갈까요.

2-1. 실측 — PageMark SNS/Threads의 8K → 3K 역설

2026-03 시점 기록입니다. PageMark의 일일 스레드 생성 파이프라인이 컨텍스트 크기 조정 실험을 거쳤습니다.

컨텍스트 토큰	rubric 평균	체감 특징
8,200	3.4	문장이 산만. 앞뒤 맥락이 섞인다.
5,500	3.7	다소 개선. 여전히 hook이 약하다.
3,100	4.1	선명. 한 가지 메시지에 집중한다.
1,800	3.8	일부 맥락 부족. 브랜드 톤이 흐려진다.

3,100 토큰이 가장 좋았습니다. 그보다 많으면 품질이 떨어지고, 그보다 적으면 톤이 흔들립니다. 최적점은 파이프라인마다 다릅니다. 하지만 거의 모든 파이프라인에서 이 공통 패턴을 관찰했습니다 — 맨 처음 설정한 컨텍스트 크기가 대체로 최적점보다 2~3배 많습니다.

“혹시 몰라서”가 원인입니다. 만들 때는 안심이 되지만 모델은 매번 그 3배의 토큰을 읽어야 합니다. 읽은 것 중 무엇을 봐야 할지 흐려집니다. 품질이 떨어집니다.

2-2. 컨텍스트 다이어트 프로토콜

해법은 다이어트입니다. 이 프로토콜의 이름은 컨텍스트 다이어트이고, 세 단계로 컨텍스트를 분류합니다.

- **필수 (must)** — 없으면 작업 자체가 불가능한 것. 브리프, 스키마, 도메인 핵심 규칙.
- **선택 (optional)** — 있으면 도움이 되지만 없어도 작동하는 것. 과거 성공 샘플 2~3개, 브랜드 톤 핵심 3줄.
- **불필요 (strip)** — 확신 없이 넣었던 것 전부. 과거 히스토리 전체, 다른 파이프라인의 지침, 경쟁사 자료.

각 조각마다 “이게 없으면 품질이 즉시 떨어지는가?”를 물어 봅니다. 답이 “확실히 예”가 아니면 일단 빼고 돌려 봅니다. 돌려 본 결과가 나쁘면 다시 넣으면 됩니다. 넣기 전에 빼 보는 방향이 다이어트의 핵심입니다.

2-3. 다이어트 실전 사례 3건

PageMark SNS 말고도 다이어트로 효과를 본 파이프라인 두 개를 더 기록합니다. 셋을 나란히 놓으면 패턴이 보입니다.

사례 A · PageMark SNS/Threads

단계	구성	토큰	rubric 평균
Before	브리프 + 브랜드 가이드 전문 + 과거 스레드 20건 + 경쟁사 스레드 10건	8,200	3.4
Diet 1	브리프 + 브랜드 3줄 + 과거 스레드 3건	3,100	4.1
Diet 2 (확인용)	브리프 + 브랜드 3줄 + 과거 스레드 0건	1,800	3.8

교훈: 과거 샘플 3건이 sweet spot. 20건은 과잉, 0건은 과소. 중간을 찾는 데 일주일 실험.

사례 B · DeckBrew Newsletter

단계	구성	토큰	rubric 평균
Before	뉴스레터 편집 가이드 40줄 + 이번 주 기사 20건 + 지난 뉴스레터 8건	12,400	3.2
Diet 1	편집 가이드 10줄 핵심 + 이번 주 기사 20건 + 지난 뉴스레터 2건	5,800	3.7
Diet 2	편집 가이드 10줄 + 기사 20건 + 지난 뉴스레터 1건	4,900	4.2

교훈: 지난 뉴스레터 1건이면 톤 일관성에 충분. 8건은 “같은 말 반복”하게 만들어 오히려 품질 저하.

사례 C · Quiet Ark Test Creation (심리테스트 문항 생성)

단계	구성	토큰	rubric 평균
Before	심리학 문헌 5편 발췌 + 기존 문항 30개 + 채점 매 뉴얼 전문	14,800	4.0
Diet 1	심리학 핵심 3개 발췌 + 문항 샘플 5개 + 채점 원칙 7줄	4,200	4.5
Diet 2	심리학 요지 1줄 + 문항 샘플 3개 + 채점 원칙 5줄	2,400	3.9

교훈: Diet 1이 최적점. Diet 2에서는 심리학 깊이가 부족해졌음. 도메인에 따라 sweet spot이 다릅니다.

세 사례의 공통 패턴: 원래 컨텍스트의 1/3 ~ 1/2 크기가 대개 최적점. 더 줄이면 도메인 특수성을 잃고, 더 두면 초점을 잃습니다.

3. P6 · 신선도 감사

다이어트가 끝난 컨텍스트도 시간이 지나면 변질됩니다. 이 변질을 감지하는 정기 점검이 P6입니다.

3-1. 실측 — Nightwatch Seed Review의 단종된 모델명

2026-01에 쓴 system prompt가 있었습니다. 2026-04에 점수가 애매하게 낮다는 것을 발견하고 system prompt를 열어 봤더니 첫 줄에 이렇게 적혀 있었습니다.

모델: <가상 모델명 V> (2025년 12월 버전)

문제는 두 가지였습니다. 첫째, 이 모델명은 SDK에서 이미 deprecated 됐고 silent fallback으로 최신 모델에 매핑되고 있었습니다. 에러는 안 났습니다. 둘째, 해당 모델의 특성에 맞춰 튜닝된 프롬프트였기 때문에 새 모델에서는 최적이지 않았습니다.

3개월간 rubric 평균이 4.3 → 4.1 → 3.9 → 3.7로 조용히 내려갔습니다. 아무도 몰랐습니다. 에러가 없었기 때문입니다.

3-2. 신선도 감사 체크리스트

분기에 한 번 (3개월마다) 이 다섯 항목을 확인합니다.

- ☐ system prompt에 적힌 모델명이 현재 지원되는 모델인가
- ☐ 예시로 쓴 URL이 아직 live인가 (HTTP 200 확인)
- ☐ 참조 파일의 디렉토리 경로가 여전히 존재하는가
- ☐ 금칙어 목록이 현재 브랜드 가이드 최신판과 맞는가
- ☐ rubric 가중치가 현재 사업 우선순위를 반영하는가

다섯 개 다 체크되면 P6 점수가 5점 이상. 셋 미만이면 3점 이하입니다.

3-3. 조용한 하락의 구조

P6 위반이 가장 위험한 이유는 사일런트이기 때문입니다. 다른 원칙들은 위반하면 뭔가 터집니다 — API 비용 폭발(P1), 깨진 콘텐츠(P3), 복구 불가(P4). 터지는 건 눈에 띕니다.

P6은 안 터집니다. 대신 1년이 지나고 돌아보면 점수가 0.5 낮아져 있는 식입니다. 하루 단위로는 관찰 불가능한 기울기. 이 기울기를 캘린더에 분기 감사 이벤트를 박아 두는 것 말고는 막을 방법이 없습니다. 이 책의 부록 C HOC 사이클 체크리스트에 P6 감사가 들어가 있는 이유입니다.

4. P2와 P6이 같이 터지는 패턴

두 원칙의 위반이 같이 발생하는 대표 패턴이 있습니다. 오래된 파이프라인에 새 컨텍스트를 계속 덧붙이는 현상입니다.

예시 시나리오. 2026-01에 만든 파이프라인. 최초 컨텍스트는 3,000 토큰으로 깔끔했습니다. 그런데 2월에 “이 브랜드 가이드도 참고해 줘” 하고 2,000 토큰 추가. 3월에 “이 경쟁사 사례도” 하고 1,500 토큰 추가. 4월에 “이 과거 히트작도 참고하면 좋을 것 같아서” 하고 2,000 토큰 추가. 총 8,500 토큰.

4월 시점에서 첫 3,000 토큰은 2026-01 버전입니다. 모델명이 단종됐을 수 있고, URL이 죽었을 수 있습니다. 나머지 5,500 토큰은 비교적 최신이지만 양이 많아서 모델이 무엇을 봐야 할지 모릅니다.

P2(양)와 P6(연식)이 동시에 문제. 이 상태의 파이프라인은 점수가 급격히 내려가지는 않습니다. 매월 0.1씩 내려갑니다. 분기 단위로 보면 0.3 하락. 1년이면 1.2 하락. 하지만 매일 보는 운영자는 내려갔다는 걸 모릅니다.

해법은 한꺼번에 두 가지 — 추가된 컨텍스트를 전부 다이어트 + 프롬프트 전문을 다시 읽고 신선도 감사. 큰 청소 작업이 됩니다. 그래서 분기마다 하는 게 좋습니다. 매년 한 번은 너무 크고, 매달 한 번은 너무 번거롭습니다.

5. 실전 — 3분 신선도 감사 루틴

매일 하는 건 아닙니다. 분기에 한 번, 3분이면 끝나는 루틴입니다.

5-1. system prompt 문단별로 시선 스캔

`prompts/gen_system.md` 와 `prompts/eval_system.md` 를 연 다음, 문단별로 연식을 느껴봅니다. “이 문단을 내가 마지막으로 건드린 게 언제지?” 기억이 안 나면 오래된 것입니다. 오래된 문단에는 ⚠ 표시. 이 단계 1분.

5-2. 모델명·URL 일괄 grep

```
# 프롬프트 파일에 들어간 모델명·URL을 뽑아서
grep -rEn 'model["\s:]=?["\s]*[a-zA-Z0-9.-]+' prompts/
grep -rEn 'https?://[^\s)]+' prompts/
```

뽑힌 목록을 현재 지원 모델 목록과 대조. URL은 `curl -I` 로 200 확인. 이 단계 1분.

5-3. rubric 가중치 비즈니스 정렬 확인

`rubric.yaml`을 열어 각 축의 가중치를 보고, 지금 이 파이프라인이 가장 중요시하는 게 이 가중치와 맞는가? 를 한 번 묻습니다. 만약 3개월 전에 “hook 가중치 0.3” 으로 시작했는데 지금은 actionability가 더 중요하다면 가중치 조정 필요. 이 단계 1분.

3분 루틴이 분기에 한 번씩만 돌아도 조용한 하락이 반 이상 차단됩니다.

6. 의심스러운 신호 · P2·P6 위반 증상

P2 위반 신호

- ☐ system prompt가 8,000 토큰 이상이다.
- ☐ 최근 한 달에 “이 맥락도 참고해 줘” 하고 두 덩어리 이상 덧붙였다.
- ☐ 같은 입력을 두 번 돌리면 결과 격차가 30% 이상 벌어진다.
- ☐ 프롬프트를 500 토큰 줄여 본 적이 없다.

P6 위반 신호

- ☐ system prompt를 마지막으로 연 것이 3개월 이전이다.
- ☐ 프롬프트 안의 모델명이 SDK의 deprecation 경고에 등장한 적 있다.
- ☐ rubric 가중치가 처음 설정한 뒤 한 번도 바뀌지 않았다.
- ☐ 참조 URL을 최근 3개월 안에 열어 본 적 없다.

각각 두 개 이상이면 해당 P 점수 3점 이하입니다.

7. 내일 30분에 할 것

- ☐ 가장 오래된 파이프라인을 고르세요. system prompt 파일을 여세요.
 - ☐ 현재 토큰 수를 측정하세요. (대충 “한글 1자 = 1.5토큰” 어림)
 - ☐ 문단을 세 단계로 분류하세요 — 필수 / 선택 / 불필요.
 - ☐ 불필요 문단을 일단 삭제하고 돌려 보세요. 결과가 나쁘면 되돌리면 됩니다.
 - ☐ 모델명·URL 한 번씩 grep해서 현재 live 여부 확인.
 - ☐ 분기 감사 일정을 캘린더에 3개월 후 예약하세요.
-

인용·참고

- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24 — P2 Context Anxiety Mitigation / P6 Harness Staleness Audit.
- Source: A. Karpathy, Context Engineering, 2026 — 선택·압축·순서·격리·포매팅 5동사.
- Source: 저자의 운영 스튜디오 Quiet Ark의 PageMark SNS / Nightwatch Seed Review 2026-01 ~ 2026-04 기록.

Ch 7 · 스코어카드 시스템 — 14 파이프라인 전수 공개

0. 이 장의 문장 하나

이 챕터가 이 책의 심장부입니다. 여기까지 읽으셨다면 여섯 개 축을 알고 네 개의 실전 원칙을 보셨습니다. 이제 그 모든 것을 한 장의 표로 합칩니다. 14개의 파이프라인에 6개의 축이 곁들인 84 개의 셀. 그 표 위에서 당신은 처음으로 자기 파이프라인을 점수로 마주하게 됩니다.

1. 14 파이프라인 카탈로그

저는 스물세 개의 파이프라인을 운영합니다. 이 책에서는 그 중 열네 개를 공개합니다. 열네 개로 줄인 이유는 세 가지입니다.

첫째, 열네 개면 S·A·B·C 네 등급이 모두 드러납니다. 둘째, 네 개의 가상 스튜디오에 깔끔하게 나뉘어 보여집니다. 셋째, 분량이 열네 개를 넘어가면 표가 스캔의 대상이 아니라 읽기의 대상이 되어 버립니다. 독자가 자기 파이프라인을 비추어 볼 때는 스캔이 맞습니다.

네 개의 가상 스튜디오는 앞 챕터에서 이미 등장했습니다. 다시 정리합니다.

스튜디오	도메인	티어
Quiet Ark	1인 운영 스튜디오 본체	—
PageMark	B2C 독서 노트·커뮤니티	Tier 1
DeckBrew	B2B SaaS 대시보드	Tier 2
Nightwatch	내부 모니터링·야간 운영	Tier 3

각 스튜디오에 14개 파이프라인이 이렇게 할당됩니다.

#	파이프라인	스튜디오	역할
1	PageMark Blog Generation	PageMark	독서 노트 롱폼 블로그 초안
2	PageMark Content Marketing	PageMark	신규 기능 런칭 카피·이메일
3	PageMark SNS/Threads	PageMark	일일 스레드 5건 생성
4	PageMark Creator Pipeline	PageMark	크리에이터 한 명 페르소나 콘텐츠
5	DeckBrew Ebook Publishing	DeckBrew	B2B 리드 매그넷 이북 집필
6	DeckBrew News Curation	DeckBrew	산업 뉴스 일일 큐레이션
7	DeckBrew Newsletter	DeckBrew	주간 고객 뉴스레터
8	DeckBrew Card News Image	DeckBrew	카드뉴스 이미지·카피
9	Nightwatch Mentoring Review	Nightwatch	멘티 제출물 야간 자동 리뷰
10	Nightwatch Seed Review	Nightwatch	초기 아이디어 품질 점검
11	Quiet Ark Test Creation	Quiet Ark	심리테스트 문항 생성
12	Quiet Ark Mobile Build	Quiet Ark	모바일 앱 자동 빌드 파이프라인
13	Quiet Ark Meme Generation	Quiet Ark	밈 콘텐츠 자동 생성
14	Quiet Ark Meme Audit	Quiet Ark	생성된 밈 품질 감사

열네 개 중 운영자가 직접 개입하지 않으면 하루 안에 품질이 떨어지는 건 다섯 개. 일주일엔 버티는 건 여섯 개. 한 달 이상 자동 운전이 가능한 건 세 개. 이 구분은 티어 경계와 일치하지 않습니다. 점수와 별개로 어느 파이프

라인이 얼마나 손이 많이 가는가는 다른 축이기 때문입니다. 책 뒷장의 부록 C HOC 사이클 체크리스트에 이 축을 추가하는 것을 권장합니다.

2. 6원칙 × 14 파이프라인 전수 공개

가중합 공식을 다시 적어 둡니다.

$$\text{가중합} = P1 \times 0.25 + P2 \times 0.15 + P3 \times 0.20 + P4 \times 0.15 + P5 \times 0.10 + P6 \times 0.15$$

그리고 이 표가 이 책의 심장입니다.

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	티어
1	PageMark Blog Generation	3	3	6	3	7	3	4.10	B
2	PageMark Content Marketing	8	5	7	6	8	2	6.30	A
3	PageMark SNS/ Threads	3	5	5	3	7	3	4.15	B
4	PageMark Creator Pipeline	4	7	8	8	8	6	6.45	A
5	DeckBrew Ebook Publishing	8	5	6	8	2	3	5.90	B
6	DeckBrew News Curation	8	7	8	7	9	2	7.00	A

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	티어
7	DeckBrew Newsletter	5	5	6	3	7	3	4.85	B
8	DeckBrew Card News Image	3	5	5	5	7	3	4.40	B
9	Nightwatch Mentoring Review	5	5	6	5	7	3	5.30	B
10	Nightwatch Seed Review	3	5	5	5	7	3	4.40	B
11	Quiet Ark Test Creation	8	7	9	9	3	2	7.00	A
12	Quiet Ark Mobile Build	8.5	8	9.5	9	8.5	7	8.48	S
13	Quiet Ark Meme Generation	1	7	1	2	1	1	2.05	C
14	Quiet Ark Meme Audit	6	5	7	7	7	2	5.70	B

2026-04-17 시점 측정값. 가중합은 소수점 둘째 자리 반올림.

3. 첫 눈에 들어오는 것

이 표를 처음 보셨다면 아마 눈이 네 곳으로 갑니다.

첫째, **12번 Quiet Ark Mobile Build 의 8.48**. 이 한 줄이 단 하나뿐인 S-tier입니다. 열네 개 중 한 개.

2026-04-17 측정 직후 저도 이 한 줄을 오래 들여다봤습니다. 모바일 앱 빌드 파이프라인이 콘텐츠 파이프라인들보다 모든 축에서 높다는 사실이 처음에는 의외였습니다. 지금은 의외가 아닙니다. 빌드 파이프라인은 결과물의 성공/실패가 명확합니다. 빌드가 되든지 안 되든지 둘 중 하나. 이 명확함이 P3(Sprint Contract)을 저절로 9.5점으로 끌어 올립니다. 파일도 명확합니다. 입력은 커밋, 출력은 .apk. P4 9점. 평가자도 분리가 쉽습니다 — 빌드 서버가 곧 평가자이고 생성자(커밋)와 완전히 독립. P1 8.5점. 이 세 개의 축이 자동으로 높으면 가중합이 올라갑니다. 명확한 도메인의 파이프라인은 이미 S-tier에 가깝게 태어납니다.

둘째, **13번 Quiet Ark Meme Generation 의 2.05**. 가장 낮은 점수입니다. Generator와 Evaluator가 같은 로컬 모델이고(P1=1), validate 스크립트가 없고(P3=1), 루브릭도 없습니다(P5=1). 하네스가 거의 비어 있는 파이프라인입니다. 이 한 줄이 책의 4장 이후 모든 챕터의 반례입니다. §5에서 이 파이프라인이 5.2로 올라가는 HOC 사이클 로그를 보여 드리겠습니다.

셋째, **P5 열의 변동폭**. 1점부터 9점까지 가장 넓게 퍼져 있습니다. P5(Value Encoding, 루브릭)는 한 번 도입하면 점수가 크게 오르고, 안 하면 1점에 머무는 이분법적 축입니다. 5번 DeckBrew Ebook Publishing이 2점인 이유도 단순합니다 — 루브릭 없이 “편집자가 보고 판단”하는 파이프라인이니까. 루브릭 하나 도입하면 그 한 줄이 B-tier에서 A-tier 초입으로 옮겨갑니다.

넷째, **P6 열의 낮은 평균**. 열네 개 중 여섯 개가 3점 이하입니다. P6(Staleness Audit)은 안 하면 아무 일도 안 일어나는 축이라서 가장 방치되기 쉽습니다. 예러가 안 나기 때문에 낮아도 문제가 없는 것처럼 느껴집니다. 사실은 조용히 품질이 내려갑니다. 6장에서 Nightwatch Seed Review의 단종된 모델명 3개월 체류 사례를 다뤘습니다.

3-B. 14 파이프라인 · 한 줄씩의 사연

표의 숫자 뒤에 사람의 이야기가 있습니다. 열네 개 파이프라인 각각의 왜 이 점수인지를 한 문단씩.

1 · PageMark Blog Generation (4.10) — 초기 B-tier의 전형입니다. rubric이 도입되면서 P5가 7점으로 올라가긴 했지만, Generator와 Evaluator가 여전히 같은 세션에서 돌고 있어 P1이 3점에 묶여 있습니다. 다음 사연을 가장 먼저 손댈 곳이 P1 분리. 그 하나만으로 가중합이 5.35로 올라갈 여지.

2 · PageMark Content Marketing (6.30) — A-tier 상위. 런칭 카피는 매번 다른 팀이 독립 리뷰하는 구조라 P1이 8점으로 높습니다. P5 루브릭도 브랜드 가이드와 잘 정렬돼 8점. 발목을 잡는 건 P6 2점 — 3개월 이상 된 과거 성공 카피가 system prompt에 그대로 남아 있어 신선도 감사가 거의 0입니다.

3 · PageMark SNS/Threads (4.15) — 전형적인 컨텍스트 과잉 파이프라인의 점수. 6장에서 8K→3K로 품질이 올라간 바로 그 파이프라인입니다. 이 측정값(4.15)은 다이어트 이전 값. 다음 분기 재측정 예상: P2 3→6, 가중합 4.15→4.60.

4 · PageMark Creator Pipeline (6.45) — 개별 크리에이터 페르소나 특화 파이프라인. 페르소나당 rubric을 따로 두는 구조라 P5가 8점. Evaluator도 페르소나 담당 에이전트가 아닌 중립 심사자가 돌아 P1 4점이 넘습니다. P2가 7로 높은 이유는 컨텍스트가 페르소나 당 압축 요약으로 최소화되어 있기 때문.

5 · DeckBrew Ebook Publishing (5.90) — B-tier 상위, A-tier 근접. P1이 8점으로 매우 높은 이유는 출간 파이프라인에 외부 편집자 레이어가 들어가 있어서. P3도 6점으로 괜찮습니다. 치명적 약점은 P5 2점 — 루브릭이 없고 편집자 주관에 의존. 8장 실패 4번의 가중치 오설정 사례가 이 파이프라인입니다.

6 · DeckBrew News Curation (7.00) — A-tier. 3장의 128→8회 사례의 주인공. Gen/Eval 분리 후 P1이 8로 상승. 뉴스 분류는 정답이 비교적 명확해 P3(9점)과 P5(9점)가 높습니다. 발목은 P6 2점 — 분류 카테고리 체계가 6개월 이상 업데이트되지 않고 있음.

7 · DeckBrew Newsletter (4.85) — 중간 B-tier. 4장의 rubric.yaml 7축이 이 파이프라인의 것. rubric이 있으니 P5가 7점. 그러나 Evaluator가 여전히 Generator와 같은 세션 — P1 5점에 묶임. 다음 사이클에 P1 분리 + P6 감사 추가하면 B-tier 상위로.

8 · DeckBrew Card News Image (4.40) — 7장 §7의 교정 중 하락 파이프라인. 루브릭을 제대로 적용하면서 허수 점수가 실수 점수로 바뀌었고, 지금 4.40이 진짜 베이스라인. P1 3점이 여전히 약점. 이미지 파이프라인은 Evaluator 분리가 텍스트보다 어려워 (이미지 평가용 멀티모달 분리) 다음 분기 작업 예정.

9 · Nightwatch Mentoring Review (5.30) — 중간 B-tier. 5장의 “운영자 머릿속 규칙” 문제가 이 파이프라인의 P4 상한선. 나머지 축은 대체로 5점 근처로 안정. [docs/issue_handling_rules.md](#) 파일 하나만 쓰면 P4가 6으로 올라 5.45.

10 · Nightwatch Seed Review (4.40) — 6장의 단종된 모델명 3개월 체류 파이프라인. P6이 3점인 현재 값도 감사 후 교정한 결과. 감사 이전에는 P6 1점에 가까웠고 rubric 점수가 조용히 4.3→3.7로 내려가고 있었음. 교정 후 재측정 기대.

11 · Quiet Ark Test Creation (7.00) — A-tier. 심리테스트 문항 생성은 문항 검증이 자동 가능한 특수 도메인. JSON 스키마 검증 + 문항 내적 일관성 검사가 validate로 들어가 P3이 9점. 파일 핸드오프도 P4 9점. 약점은 P5 3점 — 루브릭이 아직 “정답성” 중심이고 심리학적 깊이 축이 없음. 추가 예정.

12 · Quiet Ark Mobile Build (8.48) — 단 하나뿐인 S-tier. 이 파이프라인이 왜 S-tier인지는 §3에서 설명. 한 줄로 요약하면 성공/실패가 기계적으로 명확한 도메인은 하네스가 쉽게 서고, 점수가 저절로 올라간다. 교훈: 스코어카드 S-tier를 목표로 한다면 측정 가능한 성공 기준이 있는 도메인부터 접근.

13 · Quiet Ark Meme Generation (2.05) — §5의 HOC 사이클 주인공. 측정 당시 가장 낮은 점수. 하네스가 거의 비어 있었던 파이프라인. 5.20으로 올라간 후에도 P2·P4·P6은 여전히 낮음 — 다음 두 사이클의 재료.

14 · Quiet Ark Meme Audit (5.70) — B-tier 상위. 흥미로운 지점은 Meme Generation(2.05)과 Meme Audit(5.70)가 같은 도메인의 Gen/Eval 쌍 같이 움직인다는 것. Audit이 Generation의 Evaluator 역할을 담당 하는데, Audit 자체의 P1이 6점이라 자기 채점 관대함이 제한적. Generation이 Audit으로부터 받는 피드백의 엄격함이 Audit의 P1에 직접 연동됩니다.

4. 티어 분포

열네 개 파이프라인의 티어 분포는 이렇게 나옵니다.

티어	개수	비율	이 책에서의 역할
S (4.0+)	1	7%	이미 잘 돌아가는 기준점
A (3.0~4.0)	4	29%	개선 여지가 남은 상위 그룹
B (2.0~3.0)	7	50%	책이 가장 많이 다루는 중위 그룹
C (1.0~2.0)	2	14%	4장 이후의 반례 그룹
D (<1.0)	0	0%	없음
평균 가중합			5.24

평균 5.24는 스물세 개 전체 파이프라인의 평균 4.82보다 높습니다. 이 책이 의도적으로 개선 방향성이 보이는 샘플을 선별했기 때문입니다. 실제 운영에서는 열네 개의 약 절반이 C-tier이거나 D-tier 경계에 걸쳐 있을 가능성이 큼니다. 당신의 첫 측정이 평균 3점대가 나와도 놀라지 마시기 바랍니다. 저의 첫 스물세 개 측정도 4.82였고, 그 중 많은 수가 “괜찮을 것 같았는데 아니었던” 파이프라인들이었습니다.

5. HOC 사이클 — 2.05에서 5.2까지

표에서 가장 낮은 13번 Quiet Ark Meme Generation. 이 파이프라인이 세 달 만에 5.2로 올라간 기록을 HOC 사이클 네 단계로 풀어 보겠습니다. HOC 사이클은 Diagnose → Design → Implement → Verify의 네 단계이고, 각 단계의 머릿글자입니다.

Phase DA · Diagnose

첫째 주. P별 점수를 매기는 게 먼저였습니다.

```
P1 (Gen/Eval 분리) : 1 - 로컬 모델이 생성 + 자가 평가
P2 (Context 불안) : 7 - 컨텍스트가 가벼워서 P2는 높음
P3 (Sprint 계약)   : 1 - validate 스크립트 없음
P4 (파일 핸드오프) : 2 - 중간 산출물을 세션 메모리에만 저장
P5 (가치 인코딩)   : 1 - 루브릭 없음, '재밌으면 OK' 수준
P6 (신선도 감사)   : 1 - 3개월 전 프롬프트 그대로
→ 가중합 2.05 · C-tier
```

원인 진단: 거의 모든 축이 미비. 단 P2만 의외로 양호. 이 파이프라인은 하네스 없이 LLM 호출만 반복하는 상태였습니다.

Phase ID · Design

둘째 주. 어디부터 손대야 할지가 가장 어려웠습니다. 여섯 개가 다 낮는데 다 동시에 고칠 순 없습니다. 가중치 순서로 결정했습니다.

- P1을 먼저 (가중치 0.25): Evaluator를 로컬 모델에서 Claude 호출로 분리. 다른 시스템 프롬프트.
- P3을 다음 (가중치 0.20): validate.py 신설. JSON 스키마 + 길이 + 금칙어 + 품질 최소 기준.
- P5를 그다음 (가중치 0.10): rubric.yaml 5축 (hook / 구조 / 톤 / 고유성 / 길이). 가중치 합 1.0.

P2·P4·P6은 이번 사이클에서 건드리지 않기로 결정했습니다. 이유: 한 번에 세 개만 고치는 게 Verify 단계에서 어느 변화가 어느 개선을 만들었는지 귀속시키기 좋습니다. 변화 셋을 한꺼번에 넣으면 결과 셋 중 어떤 게 어느 변화 때문인지 몰라집니다.

Phase IX · Implement

3~4주. 구현 자체는 의외로 빨랐습니다. evaluator.py가 60줄, validate.py가 80줄, rubric.yaml이 40줄. 한 주면 다 썼을 양입니다. 실제로 시간을 먹은 건 rubric.yaml의 5축을 정의하는 것이었습니다. “재밌다”를 다섯 개의 측정 가능한 축으로 쪼개는 게 이 사이클에서 가장 오래 걸린 부분입니다. 결국 이렇게 정의했습니다.

```
axes:
- name: hook
  weight: 0.30
  scale:
    1: "첫 3초 안에 시선이 멎지 않는다"
    3: "절반의 독자가 두 번째 컷까지 본다"
    5: "첫 컷만으로 스크롤이 멈춘다"
# ... 나머지 4축
```

각 축에 1·3·5점의 구체적 장면을 써 두는 게 핵심이었습니다. 2점과 4점은 내삽. 이렇게 쓰지 않으면 평가자가 또다시 주관으로 돌아갑니다.

Phase VV · Verify

5~6주. 이 단계는 반드시 별도 세션에서 수행합니다. IX 단계의 기억을 가진 세션이 Verify를 하면 자기 변경을 과대평가합니다. 새 세션을 열어 변경 전 파이프라인의 산출물 30건과 변경 후 산출물 30건에 rubric을 적용했습니다.

```
P1 (Gen/Eval 분리) : 1 → 6 (+5)
P2                  : 7 → 7 (±0)
P3                  : 1 → 7 (+6)
P4                  : 2 → 2 (±0)
P5                  : 1 → 6 (+5)
P6                  : 1 → 1 (±0)
→ 가중합 2.05 → 5.20 · B-tier (C-tier 탈출)
```

세 개의 축만 건드렸는데 가중합이 세 배 이상 올랐습니다. P2·P4·P6은 다음 사이클의 과제로 남았습니다. 목표는 B-tier 안착 시점에 한 번에 한 축씩.

사이클이 끝나고 남은 것

Meme Generation의 가중합 5.20은 대단한 숫자가 아닙니다. 열네 개 평균 5.24보다 낮습니다. 하지만 이 사이클에서 진짜 가치 있었던 것은 숫자가 아닙니다. 이 파이프라인의 다음 약점이 P2·P4·P6이라는 것을 확실하게 안다는 것입니다. 다음 사이클의 설계가 정확해집니다. 감이 아니라 데이터입니다.

5-B. HOC 사이클 심화 · PageMark Blog Generation (4.10 → 5.35)

두 번째 심화 사례. 2026-03 중반에 착수한 PageMark Blog Generation 개선.

Phase DA

P1 (Gen/Eval 분리) : 3 - Evaluator가 Generator와 같은 세션
P2 (Context 불안) : 3 - 8K 컨텍스트, 개선 여지 있음
P3 (Sprint 계약) : 6 - validate.py 있음, 중간 수준
P4 (파일 핸드오프) : 3 - 중간 산출물 일부만 파일
P5 (가치 인코딩) : 7 - rubric.yaml 7축 갖춤
P6 (신선도 감사) : 3 - 분기 감사 1회 수행
→ 가중합 4.10 · B-tier

원인 진단: P5는 이미 높는데 P1이 관대함을 허용. 루브릭이 있어도 자기 평가자가 루브릭을 관대하게 적용. “이 블로그가 톤 5점이라고 평가자가 판단”하는 회로가 깨져 있음.

Phase ID

- P1 먼저 (가중치 0.25): Evaluator를 블로그 작성 권한 없는 별도 에이전트로. 시스템 프롬프트에 “당신은 편집장입니다. 작가의 의도를 존중하지 말고 독자 관점에서 읽으세요.”
- P4 다음 (가중치 0.15): 이미 파일 기반이었지만 중간 리비전이 세션 메모리에만 있었음. 각 리비전을 `runs/{hash}/02a_draft.json`, `02b_revised.json` 형태로 분리.

Phase IX

구현 기간 2주. evaluator.py 재작성 40줄, 파일 핸드오프 구조 리팩토링. 기존 산출물 20건 재검증.

Phase VV

별도 세션에서 재측정. 20건 블로그 Before/After 비교.

P1 : 3 → 6 (+3)
P2 : 3 → 3 (±0)

P3 : 6 → 6 (±0)
P4 : 3 → 5 (+2)
P5 : 7 → 7 (±0)
P6 : 3 → 3 (±0)
→ 가중합 4.10 → 5.35 · B-tier 상위

두 개 축만 건드렸는데 가중합이 1.25 올랐습니다. P1 가중치(0.25)가 크기 때문에 작은 개선도 큰 변화를 만듭니다. 다음 사이클은 P2(컨텍스트 다이어트)와 P6(분기 감사 정례화)을 목표로. A-tier 진입을 노릴 여지.

5-C. HOC 사이클 심화 · Nightwatch Seed Review (4.40 → 5.15)

세 번째 심화 사례. 6장의 단종 모델명 3개월 체류 파이프라인의 개선 사이클.

Phase DA

P1 : 3 - 같은 세션에서 시드 아이디어 생성과 평가 겸임
P2 : 5 - 컨텍스트 크기는 중간
P3 : 5 - validate 있음, 느슨함
P4 : 5 - 대부분 파일, 일부 memory
P5 : 7 - rubric 7축 갖춤
P6 : 3 - 단종 모델명 3개월 방치, 감사 없음
→ 가중합 4.40 · B-tier

특이 사항: P6이 낮는데 평가 루브릭은 7점. 모순처럼 보이지만 실제로는 루브릭 자체도 오래됐기 때문에 루브릭 점수가 낮은 기준을 관대하게 통과시키고 있었음. P6이 이중으로 영향을 미치는 패턴.

Phase ID

- P6 먼저 (가중치 0.15이지만 여기선 핵심 원인): system prompt의 모델명 최신화 + rubric 전면 검토. 3개월간 업계에서 중요도가 바뀐 항목을 반영해 가중치 재조정.
- P1 다음 (가중치 0.25): Evaluator 분리. 시드 아이디어는 “이게 될 것 같다”의 주관이 강해서 외부 평가자가 특히 중요.
- P3 재점검: rubric 변경에 맞춰 validate.py의 품질 하한도 조정.

Phase IX

3주. 모델명 교체 후 20개 시드 재평가. rubric 변경 후 다시 40개 시드 평가.

Phase VV

P1 : 3 → 5 (+2)
P2 : 5 → 5 (± 0)
P3 : 5 → 6 (+1)
P4 : 5 → 5 (± 0)
P5 : 7 → 6 (-1) ← 의도된 하락
P6 : 3 → 7 (+4)
→ 가중합 4.40 → 5.15 · B-tier 중위

P5가 -1로 내려간 것이 이 사이클의 숨은 성과입니다. 허수였던 7점이 실수 6점으로 교정됐습니다. P6 감사가 P5 허수를 드러내는 교훈적 사례. 다음 사이클은 P2·P4 개선 + P5를 다시 7로 올리기.

5-D. 세 개의 HOC 사이클을 한 장으로

파이프라인	Before	건드린 축	After	Δ	주 교훈
Quiet Ark Meme Generation	2.05	P1·P3·P5	5.20	+3.15	밑바닥에서 한 번에 세 축
PageMark Blog Generation	4.10	P1·P4	5.35	+1.25	가중치 무거운 축 우선
Nightwatch Seed Review	4.40	P6·P1·P3	5.15	+0.75	허수 교정 포함, 숫자보다 진실성

세 사례의 공통 패턴: 1. 가장 낮은 축부터가 아니라 가중치 × 개선 가능성이 가장 큰 축부터 2. 한 사이클에 2~3개 축만 건드리기 (귀속 가능성 보존) 3. Verify 단계를 반드시 별도 세션에서

다음 장에서 실패 로그를 다룰 때, 이 세 개의 성공 사례와 실패 로그의 원인 P 번호를 나란히 두면 패턴이 더 선명해집니다. 성공은 가중치 순서대로 축을 건드린 사례이고, 실패는 중요한 축을 안 건드린 채 다른 것부터 건든 사례입니다.

6. 평균 개선 — 4.82에서 5.06으로

책 프롤로그에서 “몇 달 만에 4.82에서 5.06으로” 라고 썼습니다. 그 숫자가 어떻게 나왔는지를 이 챕터 안에서 해체합니다. 전체 스물세 개 파이프라인 중 이번 사이클에서 손댄 것은 여섯 개. 나머지 열일곱 개는 측정만 하고 건드리지 않았습니다. 여섯 개의 가중합이 평균 2.8만큼 올라갔고, 열일곱 개는 그대로였습니다. 전체 평균이 $4.82 \rightarrow 5.06$ 으로 바뀌는 계산이 나옵니다.

$4.82 \times 23 = 110.86$ $110.86 + (2.8 \times 6) = 127.66$ $127.66 / 23 = 5.55\cdots$ 어라 숫자가 맞지 않습니다.

실제로는 이렇게 됩니다. 여섯 개 중 일부는 기대만큼 오르지 못했고, 한 개는 오히려 측정 시점 리스크로 점수가 내려간 것처럼 보였습니다. 이 현상을 다음 절에서 다룹니다.

7. 점수의 움직임은 언제나 순 이익이 아닙니다

사이클을 돌리는 동안 가중합이 내려간 파이프라인이 하나 있었습니다. 8번 DeckBrew Card News Image. 베이스라인 4.40에서 사이클 중반 3.90까지 떨어졌습니다. 원인은 P5 루브릭을 도입하면서 기존에 보지 못했던 약점이 드러났기 때문입니다. 루브릭 없이 “괜찮아 보이니까 7점” 이던 것이 루브릭 적용 후 “구조는 3점, 톤은 5점, 가독성은 4점” 으로 재채점되면서 평균이 내려갔습니다.

이것은 하락이 아니라 교정입니다. 이전의 7점은 허수였고, 4점이 실수였을 뿐입니다. 다만 그래프를 볼 때는 사이클 중반 떨어진 것처럼 보이고, 오너 입장에서는 개선을 했는데 점수가 내려간 불안한 순간이 됩니다. 이 순간을 지나야 진짜 점수가 드러납니다. Card News Image는 사이클 끝에 4.40으로 돌아왔습니다. 숫자는 같지만 같은 숫자가 아닙니다. 베이스라인의 4.40은 허수였고, 최종 4.40은 실수입니다.

이 패턴이 점수 측정의 두 번째 교훈입니다. 루브릭을 처음 도입한 파이프라인의 점수는 한동안 내려갑니다. 그 기간을 견디지 않으면 평가의 진실성이 회복되지 않습니다. 이 책을 읽은 뒤 첫 세 달 동안 당신의 평균 점수가 내려갈 가능성이 있습니다. 그 하락은 책이 제대로 작동하고 있다는 신호입니다.

8. 독자용 빈 스코어카드 템플릿

지금부터 이 장을 읽은 시간은 당신 것이 아닙니다. 당신 파이프라인 석 대에 이 표를 채워 넣고 나야 이 장을 읽었다고 할 수 있습니다. 부록 A에 동일한 템플릿이 다시 있습니다. 책을 덮기 전에 이 페이지를 먼저 채우시기 바랍니다.

#	내 파이프라인	P1	P2	P3	P4	P5	P6	가중합	티어
1									
2									
3									

가중합 공식을 다시 드립니다.

$$\text{가중합} = P1 \times 0.25 + P2 \times 0.15 + P3 \times 0.20 + P4 \times 0.15 + P5 \times 0.10 + P6 \times 0.15$$

계산 예시 1 · P1=3, P2=4, P3=3, P4=5, P5=2, P6=2:

$$\begin{aligned} &= 3 \times 0.25 + 4 \times 0.15 + 3 \times 0.20 + 5 \times 0.15 + 2 \times 0.10 + 2 \times 0.15 \\ &= 0.75 + 0.60 + 0.60 + 0.75 + 0.20 + 0.30 \\ &= 3.20 \rightarrow \text{A-tier 하단} \end{aligned}$$

계산 예시 2 · P1=1, P2=5, P3=1, P4=3, P5=1, P6=1:

$$\begin{aligned} &= 1 \times 0.25 + 5 \times 0.15 + 1 \times 0.20 + 3 \times 0.15 + 1 \times 0.10 + 1 \times 0.15 \\ &= 0.25 + 0.75 + 0.20 + 0.45 + 0.10 + 0.15 \\ &= 1.90 \rightarrow \text{C-tier 상단} \end{aligned}$$

첫 측정의 평균이 2점대 후반에서 3점대 중반 사이면 정상입니다. 4점대가 나오면 자기 채점을 의심하시기 바랍니다. P1 점수를 외부 세션에 재검토 요청해 보시면 자기 점수와 외부 점수의 간극을 볼 수 있습니다. 3장에서 다룬 주제가 바로 이것이었습니다.

9. 내일 30분에 할 것

- ☐ §8 빈 스코어카드에 본인 파이프라인 세 개를 적으세요. 가장 오래된 것 하나, 가장 자주 돌아가는 것 하나, 가장 의심스러운 것 하나.
 - ☐ 각 파이프라인의 P1~P6을 1~5점으로 채우세요. 확신이 없으면 3점.
 - ☐ 계산기 또는 스프레드시트로 가중합을 계산. 소수점 둘째 자리까지.
 - ☐ 세 숫자를 적어 두세요. 석 달 뒤 다시 측정할 때 증가분이 이 책이 당신에게 남긴 것입니다.
 - ☐ 석 달 뒤 측정 일정을 캘린더에 예약하세요. 3개월 + 하루 여유.
-

인용·참고

- Source: 저자의 운영 스튜디오 Quiet Ark의 2026-04-17 시점 스코어카드 측정값 (실측 23개 파이프라인 중 14개 선별 + 가상 도메인 익명화 재구성).
- Source: HOC 사이클(Diagnose → Design → Implement → Verify) 4단계 구조는 저자가 운영 중인 파이프라인 개선 사이클에서 정립한 프로토콜입니다.
- Source: P. Rajasekaran, Harness design for long-running application development, Anthropic Engineering, 2026-03-24 (가중치 개념의 원출처).

Ch 8 · 실패 로그 — 점수 몇 점에서 무엇이 터졌나

0. 이 장의 문장 하나

파이프라인은 실패할 때 어느 축이 낮아서 실패합니다. 실패를 P 번호로 귀속시킬 수 있으면 재발 방지가 가능해집니다. 이 장은 저자가 겪은 다섯 건의 실패를 원칙 번호로 해부합니다.

1. 실패 로그 템플릿

하나의 실패를 4칸으로 정리합니다.

칸	내용
사건	언제, 어느 파이프라인에서 무슨 일이 일어났는가
원인	어느 P 번호가 약해서 발생한 문제인가
해결	그때 어떻게 막았는가
방지	다시는 안 나게 어떤 구조를 넣었는가

이 4칸은 부록 B에 빈 템플릿으로 다시 수록됩니다. 아래 다섯 건은 모두 이 템플릿으로 정리됩니다.

2. 실패 1 · DeckBrew News Curation · 재생성 루프 \$45/일

사건

2026-02-08부터 2026-02-21까지 2주간, DeckBrew News Curation 파이프라인의 Anthropic API 비용이 평소 \$3/일에서 \$45/일로 15배 증가했습니다. 이상을 눈치 챈 건 2026-02-22 월간 결제 알림이 왔을 때였습니다. 2주간 누적 비용이 예산을 초과했습니다.

원인

P1 위반. Generator와 Evaluator가 같은 세션에서 돌아가고 있었습니다. 평소에는 자기 평가자가 관대해서 대부분 PASS로 넘어갔는데, 2월 초에 데이터 소스 특성이 일시적으로 바뀌자 자기 평가자가 혼란에 빠져 REJECT를 자주 냈습니다. REJECT가 나면 같은 세션이 같은 약점으로 재생성을 반복. 한 건당 128회 API 호출. 14일간 누적으로 하루 \$45.

해결

2026-02-22 낮에 파이프라인을 중지시키고 재시작 전 Evaluator를 분리했습니다. 별도 파일 `evaluator.py`, 별도 세션, 별도 시스템 프롬프트. 재시작 후 한 건당 API 호출이 128회에서 8회로 떨어졌습니다. 비용이 \$45에서 \$3으로 복귀.

방지

- Evaluator 분리 이후 Quiet Ark의 모든 신규 파이프라인 템플릿에 Generator/Evaluator 별도 파일 구조를 기본값으로 박았습니다.
- 파이프라인마다 일일 API 비용 알림을 예산의 150% 시점에 발송하도록 설정. 다음에 비슷한 이상이 일어나면 2주가 아니라 하루 안에 감지됩니다.
- 분기 감사 체크리스트에 “Generator와 Evaluator가 별도 파일·세션인가?” 를 맨 위에 추가.

3. 실패 2 · PageMark Blog Generation · 깨진 콘텐츠 2주 노출

사건

2025-11-18에 배포한 PageMark 블로그 초안이 JSON 필드 하나가 깨진 채로 CMS에 자동 업로드됐습니다. 제목 필드에 마크다운 따옴표가 escape되지 않아 CMS에서 제목이 잘려서 표시됐습니다. “어떤 책은 당신을...”까지만 보이고 나머지가 잘렸습니다. 이 상태로 2주 노출. 11월 30일 독자 피드백으로 발견. 그 사이 해당 블로그 조회수가 평소의 30%로 떨어졌습니다.

원인

P3 위반. validate.py가 존재하지 않았습니다. LLM이 돌려 준 output.json이 곧바로 CMS 업로드 API로 전달됐습니다. 스키마 검증이 한 줄도 없었기 때문에 제목 필드가 이스케이프되지 않은 채로 통과했습니다. 이 결함은 엄밀히 말하면 JSON 스키마 위반이 아니라 문자 레벨 이스케이프 누락인데, 간단한 `json.dumps(obj, ensure_ascii=False)` 후 `len(title) == len(title.strip())` 확인만 있었어도 잡을 수 있었습니다.

해결

2025-11-30 저녁에 파이프라인 중지, 깨진 게시물 전량 삭제 후 재생성, CMS 재배포. 재생성 시 임시로 제목만 수동 확인을 거쳤습니다.

방지

- 2025-12 주 안에 `validate.py` 초안 작성. JSON 스키마 + 길이 + 금칙어 + 품질 하한 4종.
- 제목 필드에 특화된 문자 레벨 검사 추가 — 마크다운 특수 문자의 escape 여부.
- 모든 파이프라인의 CMS 업로드 단계에 `validate.py` PASS 여부를 전제 조건으로 묶었습니다. PASS 아니면 업로드 차단.
- 분기마다 지난 분기에 업로드된 산출물 5건을 샘플링하여 `validate` 재검사. 과거 누락이 있었는지 감지.

4. 실패 3 · PageMark SNS/Threads · 컨텍스트 과잉으로 품질 하락

사건

2025-12 초 PageMark SNS 파이프라인의 rubric 평균이 4.0에서 3.4로 내려갔습니다. 운영자는 “더 좋은 콘텐츠를 만들려면 더 많은 맥락이 필요하다”고 판단해 과거 성공 스레드 20건을 system prompt에 추가했습니다. 컨텍스트가 5,200 토큰에서 8,200 토큰으로 증가. 품질은 계속 떨어졌습니다. 12월 말 3.4까지 하락.

원인

P2 위반. 컨텍스트 불안이 품질을 떨어뜨리는 전형적 패턴. 더 넣을수록 모델이 무엇을 봐야 할지 흐려지는 역설. “과거 성공 스레드 20건”은 도움이 아니라 소음이 됐습니다. 모델은 각 스레드의 공통점이 아니라 각 스레드의 고유한 표현을 따라 하려 했고, 결과적으로 아무 스레드도 답지 않았습니다.

해결

2026-01-10에 컨텍스트 다이어트 프로토콜 적용. 20건을 3건으로 줄이고, 브랜드 가이드 전문을 3줄 요약으로 교체. 컨텍스트가 8,200 토큰에서 3,100 토큰으로 내려갔습니다. 같은 주에 rubric 평균이 3.4에서 4.1로 반등.

방지

- 신규 파이프라인 초기 설정 단계에 컨텍스트 상한 5,000 토큰을 기본값으로 강제.
 - 컨텍스트에 추가할 때마다 “이것 없이 돌려봤는가?” 를 묻는 템플릿 주석을 system prompt 맨 위에 삽입.
 - 분기 감사에서 컨텍스트 토큰 수의 전 분기 대비 증감을 로그. +20% 이상 증가 시 다이어트 검토.
-

5. 실패 4 · DeckBrew Ebook Publishing · rubric 가중치 오설정

사건

2026-01에 DeckBrew Ebook Publishing 파이프라인의 rubric.yaml이 가중치 합이 1.2인 채로 6주간 운영됐습니다 (`actionability: 0.4` 오설정 — 정상은 0.2). 6주간 A-tier(4.5) 판정이 찍혀 있었습니다. 외부 평가자가 본 실제 품질은 C-tier 수준이었습니다. 3월에 번들 상품 출시 과정에서 외부 QA를 통해 발견.

원인

P5 위반 + 구성 오류. 루브릭 자체는 존재했으나 가중치 합 검증이 없었습니다. 수학적으로 가중 평균이 4.5에 달했지만 실제 사용된 가중치가 1.2였기 때문에 정상 스케일(0~5)을 벗어난 허수 점수였습니다. 점수의 형태만 정상이고 의미가 깨진 상태.

해결

2026-03에 가중치 합을 1.00으로 정정. 재측정 시 A-tier 판정이 B-tier 중위(3.2)로 내려왔습니다. 동시에 3부작 중 2권 『Claude Code로 1인 개발팀 만들기』의 부록 검증을 재실행해 실제 품질 확인. B-tier의 품질이지만 독자 기대치에 부합한다고 판단해 출시 강행.

방지

- 모든 rubric.yaml에 가중치 합 검증 스크립트를 `validate.py` 최상단에 배치.

```
assert abs(sum(a["weight"] for a in rubric["axes"]) - 1.0) < 1e-6, \
    "rubric weight sum is not 1.0"
```

- 루브릭 변경 시 사전 리뷰어 1인 체크 필수. “합이 1인가?” 를 눈으로 한 번 더 확인.
- 분기 감사에 rubric 메타 검증 단계 추가 — 가중치, 축 개수, 1·3·5점 정의 존재 여부.

6. 실패 5 · Nightwatch Mentoring Review · 핸드오프 파일 누락으로 12시간 중단

사건

2026-03-04 새벽. 80명 멘티 제출물을 처리하던 Nightwatch Mentoring Review 파이프라인이 6시간 지점에서 OOM으로 다운됐습니다. 파이프라인 로그가 세션 메모리에만 존재했기 때문에 어디까지 처리됐는지 알 수 없었습니다. 처음부터 재시작. 아침 8시 수업 시작 전 리뷰 전달이 목표였는데 여섯 시간 지연. 당일 멘티 중 23명이 리뷰 없이 수업에 참여.

원인

P4 위반. 핸드오프가 파일 기반이 아니었습니다. 각 멘티 제출물이 순차적으로 처리되면서 결과가 Python 리스트 한 변수에 누적되고 있었습니다. 프로세스가 죽으면 리스트도 같이 증발. 재개 기준이 없어 처음부터 재시작 외의 선택이 없었습니다.

해결

3월 4일 당일은 재시작을 선택했습니다. 하지만 3월 5일부터 파이프라인 구조 변경에 착수. 각 멘티 제출물을 `runs/{id}/01_submission.json`, `02_review.json` 형태로 디스크에 저장하는 구조로 재작성. 3월 8일 배포 완료.

방지

- 재개 가능성 테스트: 의도적으로 중간에 프로세스를 죽이고 재시작했을 때 완성되는지 를 파이프라인 배포 검수 항목에 추가.
 - 구조 변경 후 3월 10일 테스트 실행 중 의도적 중지 — 정상적으로 재개 확인.
 - 다른 Nightwatch 계열 파이프라인(Seed Review)에도 동일 구조 전파.
 - 매월 첫 주에 재개 드릴을 돌림 — 돌아가는 파이프라인 중 하나를 임의로 중지시키고 자동 재개를 확인.
-

7. 실패 6 · Quiet Ark Meme Generation · 자기 평가 허수 루프

사건

2025-12에 Quiet Ark Meme Generation 파이프라인의 일일 산출물 수가 70건에서 180건으로 급증했습니다. 모든 산출물이 자기 평가 4.7점으로 PASS였습니다. 운영자는 생산성이 올랐다고 착각해 게시 빈도를 그대로 올렸습니다. 12월 중순 Threads 엔게이지먼트가 이전 평균의 40%로 떨어진 것을 발견. 외부 평가자에 의한 재채점 결과 평균 1.9점이었습니다.

원인

P1 + P5 복합 위반. 자기 평가자가 관대한데다가(P1) 루브릭도 “재미있으면 OK”라는 단일 기준(P5)이어서 숫자가 의미를 완전히 잃었습니다. 180건 생성이 속도였지 품질이 아니었습니다.

해결

12월 말 파이프라인 중지 후 Meme Audit 에이전트를 별도 구축 (현 스코어카드의 14번 파이프라인). Generator와 Audit이 다른 세션·다른 모델·다른 시스템 프롬프트로 완전 분리. 12월 31일 재개 후 산출물 수가 70건/일로 감소했으나 외부 엔게이지먼트는 이전 평균으로 회복.

방지

- 자기 점수 4점 이상이 3일 연속 이어지면 자동 경고가 뜨도록 모니터링 추가. 관대함 감지 조기 경보.
 - 새 파이프라인 초기에 반드시 Audit 에이전트부터 만들고 Generator 연결. 순서가 반대면 속도에 취해 품질을 잃음.
 - 분기 감사에 외부 샘플 재채점을 포함. 자기 점수와 외부 점수 간극이 1점 이상이면 P1 재점검.
-

8. 실패 7 · DeckBrew News Curation · 카테고리 체계 노후화

사건

2026-03 중반, DeckBrew News Curation 파이프라인의 일일 출력에서 “기타” 카테고리 비중이 평소 8%에서 34%로 증가했습니다. 이상을 발견한 건 구독자 1명이 “모든 뉴스가 기타로 분류되는 것 같다”는 피드백을 남긴 뒤였습니다. 지난 6개월간 카테고리 체계는 한 번도 업데이트되지 않은 상태. 새로 등장한 주제 5~6개가 어느 카테고리에도 맞지 않아 기타로 몰렸습니다.

원인

P6 위반. 카테고리 체계(22개)가 2025-09에 정해진 후 업데이트되지 않은 채 2026-03까지 운영됐습니다. 신선도 감사가 없었기 때문에 기타 비중의 증가를 누구도 감지하지 못했습니다. 6장에서 경고한 조용한 하락의 전형.

해결

2026-03 말에 신선도 감사 1회차 실시. 지난 6개월 기타 분류 건을 검토해 새 카테고리 3개 (MCP·에이전트·하네스) 신설. 기존 카테고리 중 사용 빈도 0인 것 2개 제거. 재분류 후 기타 비중 11%로 정상화.

방지

- 분기마다 카테고리 분포 리포트를 자동 생성. 특정 카테고리의 월간 증감이 $\pm 15\%$ 를 넘으면 알림.
- 신규 키워드(월간 topN 단어 중 카테고리에 안 들어가는 것)를 자동 탐지하는 스크립트 추가.
- 부록 C HOC 사이클 체크리스트 §C-5에 “카테고리 체계 신선도” 항목을 분기 감사에 포함.
- 같은 패턴을 Seed Review, Mentoring Review 계열 파이프라인에도 적용 — 이들도 분류 기준이 암묵적으로 고정되어 있어 같은 위험.

9. 일곱 건을 한 장으로 — P 번호별 빈도표

일곱 건의 실패가 P 번호별로 어느 축에 집중돼 있는지.

P 번호	위반 건수	주요 사건
P1	2	재생성 루프 \$45/일 · 자기 평가 허수 루프 180건
P2	1	컨텍스트 과잉 품질 하락
P3	1	깨진 콘텐츠 2주 노출
P4	1	12시간 중단
P5	2	rubric 가중치 오설정 · 자기 평가 허수 (P1과 복합)
P6	1	카테고리 체계 6개월 노후화

P1이 두 번, P5가 두 번(한 번은 P1과 복합). 가중치가 가장 무거운 P1이 실패도 가장 많이 내는 것은 우연이 아닙니다. P1은 모든 원칙의 진실성을 담보하는 축이기 때문에 P1이 뚫리면 다른 축의 점수도 거짓으로 찍히면서 2차 실패를 유발합니다. 실패 6번이 정확히 그 패턴 — P1 위반이 P5 허수를 만들었습니다.

P6이 처음(실패 5건 시점)에는 0건이었는데 6·7번 확장 분석에서 1건 추가됐습니다. 조용한 하락은 기록으로 남기 어렵기 때문에 실패 로그에서 과소 대표됩니다. 스코어카드의 P6 평균(14개 중 6개가 3점 이하)을 실패 로그와 비교해 보면, 실제 P6 위반은 로그된 수의 몇 배가 된다고 추정하는 게 현실적입니다.

10. 실패를 P 번호로 귀속시키는 연습

당신도 지난 3개월 안에 실패를 겪으셨을 겁니다. 아래 템플릿에 한 건을 직접 써 넣으세요. 부록 B에 동일한 빈 템플릿이 다시 있습니다.

칸	내용
사건	
원인	P_
해결	
방지	

원인 칸에 P 번호를 붙이는 순간, 다음에 똑같은 유형의 실패가 터지기 전에 당신의 스코어카드에서 경고가 나옵니다. P 번호는 단순한 라벨이 아니라 조기 경보의 단위입니다.

11. 내일 30분에 할 것

- ☐ 지난 3개월 안에 파이프라인이 이상 동작했던 사건 하나를 떠올리세요.
 - ☐ 위 §10 템플릿을 채우세요. 사건·원인(P 번호)·해결·방지.
 - ☐ 사건을 P 번호에 귀속시키는 과정에서 당신 스코어카드의 해당 P 점수가 실제로 낮는지 확인하세요.
 - ☐ 낮으면 맞게 매긴 것입니다. 높는데 실패가 났다면 자기 채점의 관대함을 의심하세요.
 - ☐ 방지 칸에 쓴 내용을 이번 분기 안에 실행할 1개 액션으로 줄이세요. 달력에 기록.
-

인용·참고

- Source: 저자의 운영 스튜디오 Quiet Ark의 2025-11 ~ 2026-03 파이프라인 실패 기록 (가상 도메인 DeckBrew / PageMark / Nightwatch 로 재구성).
- Source: SRE Post-mortem 템플릿의 4칸 구조(사건·원인·해결·방지) 일반 관행.

Ch 9 · 에필로그 — 점수가 있는 파이프라인으로

책을 덮기 전에 한 가지만 확인하려 합니다.

부록 A의 빈 스코어카드가 당신 파이프라인 이름으로 채워져 있을까요?

숫자가 3점·4점·2점, 어떤 숫자든 적혀 있다면 당신은 이미 이 책을 소화한 독자입니다. 비어 있다면 이 책은 당신의 책이 아니라 저의 책으로 끝나게 됩니다. 저자가 받은 책이 아니라 독자가 받은 책이 되려면 숫자가 적혀야 합니다.

이 책이 약속한 것은 하나였습니다. 당신의 파이프라인에 점수가 쓰이게 되는 것. 점수는 정답이 아닙니다. 방향입니다. 4.2점과 4.3점의 차이는 중요하지 않습니다. 점수가 있는 파이프라인과 점수가 없는 파이프라인의 차이가 전부입니다. 전자는 다음 달 어디를 건드릴지를 알고 있습니다. 후자는 모릅니다.

모델은 곧 바뀝니다. 3개월 뒤에는 오늘 쓰는 모델보다 더 좋은 게 나옵니다. 그런데 당신이 쌓은 여섯 개의 축은 그대로 남습니다. 새 모델에 같은 점수를 매겨서 비교하면 됩니다. 당신이 쌓은 것은 모델이 아니라 점수표였습니다. 이 점은 3개월 뒤에 드러날 것입니다.

세 달 뒤에 다시 측정하세요. 이 책을 덮은 지 정확히 90일 뒤. 캘린더에 예약하셨을 겁니다. 안 하셨다면 지금 예약하시기 바랍니다. 지금, 책을 다 덮기 전에.

재측정 결과는 세 방향 중 하나입니다.

- **점수가 올랐다.** — 정확히 어느 원칙 덕분에 올랐는지 쓰세요. 그 원칙이 당신의 강점입니다. 다음 분기에도 이 방향으로.
- **점수가 같다.** — 3개월 동안 구조가 안 바뀐 것입니다. 무엇이 당신을 바꾸지 못하게 막았는지 적으세요. 시간 부족인가요, 우선순위 이탈인가요, 측정 주체의 부재인가요.
- **점수가 내려갔다.** — 4장과 7장에서 다룬 루브릭을 제대로 쓰기 시작했을 때의 교정일 수 있습니다. 허수 점수가 실수 점수로 바뀌는 건강한 하락입니다. 6장의 조용한 하락이면 P6 감사를 돌리세요.

셋 중 어느 쪽이든 다음 분기의 행동이 정해집니다. 이것이 점수표가 당신에게 주는 것입니다. 점수 자체가 아닙니다. 점수의 움직임을 읽을 수 있는 눈입니다.

앞의 두 권의 저자는 WHY와 HOW를 쓰고 자리를 비웠습니다. 이 책의 저자도 MEASURE를 쓰고 자리를 비웁니다. 3부작이 끝났다는 뜻이 아닙니다. 앞으로의 측정이 당신 차례라는 뜻입니다.

당신이 다음에 쓸 것은 당신 파이프라인의 스코어카드 히스토리입니다. 분기마다 한 줄씩 늘어날 표. 2026 Q2 시점 가중합, Q3 시점 가중합, Q4 시점 가중합. 오른쪽 끝에 '+/-' 컬럼이 있어 분기별 움직임이 보입니다. 1년이 지나면 네 줄. 3년이 지나면 열두 줄. 이 표가 당신이 운영자로서의 경력에 쌓이는 가장 정직한 이력서입니다.

제 스프레드시트는 지금 2026-04-17 버전까지 기록되어 있습니다. 다음 분기인 2026-07-17에 다시 열어 새 줄을 채울 것입니다. 그때 기록될 숫자가 얼마일지는 저도 아직 모릅니다. 다만 그 숫자가 어디서 왔는지를 정확히 쓸 수 있을 것이라는 건 압니다. 이 책이 당신에게 남기는 것도 정확히 그것입니다.

마지막으로 한 가지 부탁이 있습니다.

3개월 뒤 재측정 때, 점수가 어떻게 바뀌었든 한 줄을 저에게 들려 주세요. 어떻게 전하는가는 중요하지 않습니다. 블로그 글이어도 되고, 당신 팀의 내부 문서 한 줄이어도 됩니다. 어느 원칙이 올랐고 어느 원칙이 내려갔는지, 그리고 왜 그랬는지를 당신의 말로 써 주세요. 제가 그걸 읽지 못해도 상관없습니다. 당신이 쓰는 행위 자체가 이 책의 진짜 에필로그입니다. 저자의 에필로그가 아니라, 독자의 에필로그. 당신이 덧붙이는 장이 이 책의 마지막 장입니다.

이 책은 당신의 파이프라인에 점수가 쓰이게 되는 것을 약속했습니다. 그 약속이 지켜지려면 한 가지가 더 필요합니다. 점수를 보고 무엇을 할지를 당신이 결정해야 합니다. 책이 대신 결정해 주지 않습니다. 점수표가 대신 결정해 주지도 않습니다. 스코어카드는 지도일 뿐, 걸어갈 길을 정하는 건 항상 걷는 사람의 몫입니다.

당신의 파이프라인은 이제 몇 점일까요?

이 질문에 당신이 답할 수 있게 되었다면, 이 책의 역할은 끝났습니다.

인용·참고

- Source: 이 책 프롤로그의 첫 문장 “지금 당신이 운영 중인 AI 파이프라인은 몇 점일까요?” 와 쌍 이 루는 에필로그 마지막 문장.
- Source: 저자의 운영 스튜디오 Quiet Ark의 2026-04-17 시점 스코어카드 기록. 다음 기록은 2026-07-17 예정.

부록 A · 빈 스코어카드 템플릿

책을 덮기 전에 여기에 당신 파이프라인 세 개의 점수를 적으세요. 가장 오래된 것 하나, 가장 자주 돌아가는 것 하나, 가장 의심스러운 것 하나.

A-1. 6원칙 참조표 (가중치 포함)

ID	원칙	가중치
P1	Generator-Evaluator Separation	0.25
P2	Context Anxiety Mitigation	0.15
P3	Sprint Contract (Verification Contract)	0.20
P4	File-Based Handoff	0.15
P5	Grading Criteria Value Encoding	0.10
P6	Harness Staleness Audit	0.15
합계		1.00

A-2. 가중합 공식

$$\text{가중합} = P1 \times 0.25 + P2 \times 0.15 + P3 \times 0.20 + P4 \times 0.15 + P5 \times 0.10 + P6 \times 0.15$$

티어 경계:

- **S-tier:** 4.0 이상 5.0 이하

- A-tier: 3.0 이상 4.0 미만
- B-tier: 2.0 이상 3.0 미만
- C-tier: 1.0 이상 2.0 미만
- D-tier: 1.0 미만

A-3. 빈 스코어카드 (3 파이프라인, Q1 측정)

측정 일자: _____

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	티어
1									
2									
3									

A-4. 빈 스코어카드 (재측정, Q2 이후)

측정 일자: _____

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	전분기比	티어
1										
2										
3										

A-5-pre. 빈 스코어카드 (Q3 재측정)

측정 일자: _____

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	전분기比	티어
1										
2										
3										

A-6-pre. 빈 스코어카드 (Q4 재측정)

측정 일자: _____

#	파이프라인	P1	P2	P3	P4	P5	P6	가중합	전분기比	티어
1										
2										
3										

A-5. 계산 예시

예시 1 — P1=3, P2=4, P3=3, P4=5, P5=2, P6=2

$$\begin{aligned}
 &= 3 \times 0.25 + 4 \times 0.15 + 3 \times 0.20 + 5 \times 0.15 + 2 \times 0.10 + 2 \times 0.15 \\
 &= 0.75 + 0.60 + 0.60 + 0.75 + 0.20 + 0.30 \\
 &= 3.20 \rightarrow \text{A-tier 하단}
 \end{aligned}$$

예시 2 — P1=1, P2=5, P3=1, P4=3, P5=1, P6=1

$$\begin{aligned} &= 1 \times 0.25 + 5 \times 0.15 + 1 \times 0.20 + 3 \times 0.15 + 1 \times 0.10 + 1 \times 0.15 \\ &= 0.25 + 0.75 + 0.20 + 0.45 + 0.10 + 0.15 \\ &= 1.90 \rightarrow \text{C-tier 상단} \end{aligned}$$

부록 B · 실패 로그 템플릿

지난 3개월 안에 파이프라인이 이상 동작한 사건 한 건을 골라 아래 4칸을 채우세요. 여분으로 세 장이 준비돼 있습니다.

B-1. 실패 로그 1

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__
해결	
방지	

B-2. 실패 로그 2

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__
해결	
방지	

B-3. 실패 로그 3

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__
해결	
방지	

B-4. 실패 로그 4

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__
해결	
방지	

B-5. 실패 로그 5

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__
해결	
방지	

B-6. 실패 로그 6

사건 일자: _____ 파이프라인: _____

칸	내용
사건	
원인	P__

칸	내용
해결	
방지	

부록 C · HOC 사이클 체크리스트

Harness Operations Cycle의 네 단계. 한 파이프라인 개선에 3~6주 걸립니다. 체크리스트를 복사해 각 사이클마다 하나씩 씁니다.

C-1. Phase DA · Diagnose (1~2주)

- ☐ 파이프라인 이름·도메인을 1줄로 기록
- ☐ 현재 P1~P6 점수를 1~5점으로 기록 (확신 없으면 3점)
- ☐ 가중합 계산
- ☐ 현 티어 기록
- ☐ 가장 낮은 원칙 3개를 표시
- ☐ 각 원칙이 왜 낮은지 1줄씩 원인 기록

C-2. Phase ID · Design (1주)

- ☐ DA의 낮은 3개 중 가중치 순서로 상위 2개만 선정 ($P1 > P3 > P2 \cdot P4 \cdot P6 > P5$)
- ☐ 선정된 2개에 대한 구체 변경 사항 기록 (어떤 파일을 어떻게)

- ☐ 예상 점수 변화 기록 (ex: P1 1 → 6, P3 2 → 5)
 - ☐ 변경하지 않을 원칙들을 이유와 함께 기록 (혼란 방지)
-

C-3. Phase IX · Implement (2~3주)

- ☐ ID의 변경 사항을 코드로 반영
 - ☐ 변경된 파일 경로 기록
 - ☐ 3건 이상 실제 산출물로 sanity check
 - ☐ 배포 전 이전 설정 백업
-

C-4. Phase VV · Verify (1주)

- ☐ 별도 세션에서 재측정 (IX 세션과 분리 필수)
 - ☐ 변경 전·후 산출물 각 30건에 rubric 적용
 - ☐ P1~P6 재점수화
 - ☐ 가중합 변화 기록
 - ☐ 예상 변화와 실제 변화의 차이 분석
 - ☐ 다음 사이클에서 건드릴 원칙 지정
-

C-5. 분기 감사 체크리스트 (Phase 끝나고 3개월 뒤)

- ☐ P6 신선도 감사 5종 (§6장)
- ☐ 컨텍스트 토큰 수 전 분기 대비 증감
- ☐ rubric 가중치 합 1.0 검증

☐ 다음 HOC 사이클 시작 일자 캘린더 예약

C-6. 두 번째 HOC 사이클용 빈 로그 (사본 A)

파이프라인: _____ 사이클 시작 일자: _____

DA

원칙	점수	원인
P1		
P2		
P3		
P4		
P5		
P6		
가중합		현 티어:

ID (건드릴 축 · 가중치 순)

- _____: _____ (예상 점수 변화: __ → __)
- _____: _____ (예상 점수 변화: __ → __)

IX (파일 경로)

- _____
- _____

VV (별도 세션, Before/After)

원칙	Before	After	Δ
P1			
P2			
P3			
P4			
P5			
P6			
가중합			

다음 사이클에서 건드릴 축

C-7. 세 번째 HOC 사이클용 빈 로그 (사본 B)

파이프라인: _____ 사이클 시작 일자: _____

DA

원칙	점수	원인
P1		
P2		
P3		
P4		

원칙	점수	원인
P5		
P6		
가중합		현 티어:

ID (건드릴 축 · 가중치 순)

- _____ : _____ (예상 점수 변화: _ → _)
- _____ : _____ (예상 점수 변화: _ → _)

IX (파일 경로)

- _____
- _____

VV (별도 세션, Before/After)

원칙	Before	After	Δ
P1			
P2			
P3			
P4			
P5			
P6			
가중합			

다음 사이클에서 건드릴 축

부록 D · “내일 30분” 전체 체크리스트

각 장 말미의 블록을 한 페이지로 모았습니다. 10일 안에 이 목록을 다 체크하면 당신의 파이프라인 석 대는 이미 스코어카드를 가진 상태가 됩니다.

D-1. Day 1 (Ch1)

- ☐ 운영 중인 AI 파이프라인 전수 목록화
 - ☐ 각 파이프라인 옆에 운영 기간과 체감 품질 1줄
 - ☐ 가장 의심스러운 파이프라인 표시
-

D-2. Day 2 (Ch2)

- ☐ 6원칙 표(이름·가중치·한 줄)를 종이에 손으로 한 번 그리기
 - ☐ 표시된 파이프라인 하나에 P1~P6 점수 채우기
 - ☐ 가중합 계산
-

D-3. Day 3 (Ch3)

- ☐ Evaluator를 Generator와 다른 파일로 분리
 - ☐ Evaluator 시스템 프롬프트에 “5점은 드뭅니다” 추가
 - ☐ 다음 날 파이프라인 돌려 자기점수 vs 외부점수 간극 체감
-

D-4. Day 4 (Ch4)

- ☐ validate.py 30줄 (schema + length 최소)
 - ☐ rubric.yaml 3축 (structure/evidence/tone) 1·3·5점 구체 장면
 - ☐ 가중치 합 1.0 확인
 - ☐ 최근 3건 산출물에 적용
-

D-5. Day 5 (Ch5)

- ☐ 에이전트 간 통신 경로 종이에 그림
 - ☐ 각 화살표 위에 파일 경로 또는 “비파일” 표시
 - ☐ 비파일 하나를 파일로 교체
-

D-6. Day 6 (Ch6)

- ☐ 가장 오래된 파이프라인 system prompt 토큰 수 측정
 - ☐ 문단을 필수/선택/불필요 3단계 분류
 - ☐ 불필요 문단 일단 삭제 후 돌리기
-

- ☐ 모델명·URL grep으로 live 확인
 - ☐ 분기 감사 일정 캘린더 예약 (3개월 뒤)
-

D-7. Day 7 (Ch7)

- ☐ 본인 파이프라인 3개 빈 스코어카드에 가중합 계산
 - ☐ 세 숫자 기록
 - ☐ 3개월 뒤 재측정 캘린더 예약 + 하루 여유
-

D-8. Day 8 (Ch8)

- ☐ 지난 3개월 내 사건 1건 4칸 템플릿 채우기
 - ☐ 원인에 P 번호 귀속
 - ☐ 방지 칸의 내용을 1개 액션으로 줄여 달력 기록
-

D-9. Day 9 (Ch9 · 에필)

- ☐ 3개월 뒤 스코어카드 재측정이 캘린더에 확정됐는지 확인
 - ☐ 부록 A-3 빈 템플릿에 이번 분기 점수 기록 확정
-

D-10. Day 10 (여유일)

- ☐ 여유. 7일간 누락된 블록 보강.

☐ 이 책을 처음 읽기 시작한 날을 다이어리에 적어 두기. 3개월 뒤 재측정 때 “얼마나 변했나”의 기준점이 됩니다.

부록 E · 인용·참고 문헌

이 책 본문에서 인용한 원출처를 한 페이지로 모읍니다.

E-1. Anthropic Engineering

- **P. Rajasekaran**, Harness design for long-running application development, Anthropic Engineering, 2026-03-24.
 - 6원칙의 이름이 여기서 왔습니다. 번호와 가중치는 이 책의 저자 스튜디오가 운영 중 정립.
- **Anthropic Engineering**, Effective harnesses for long-running agents.
 - Rajasekaran 논문의 선행 포스트. Initializer + Coding agent 2에이전트 구조.

E-2. A. Karpathy

- **A. Karpathy**, Software 3.0, YC AI Startup School, 2025-06.
 - “LLM은 새로운 OS” 메타포. 3부작 전체의 배경.
- **A. Karpathy**, LLM Wiki Concept, 2026-04.
 - 파일이 영속 지식의 최소 단위라는 관점. Ch5 P4 파일 핸드오프의 사상적 배경.
- **A. Karpathy**, Context Engineering, 2026.
 - 선택·압축·순서·격리·포매팅 5동사. Ch6 P2 컨텍스트 다이어트 프로토콜의 원형.

E-3. 3부작 시리즈

- 『팀이 필요하다. 사람 말고.』 — 하네스의 WHY.
- 『Claude Code로 1인 개발팀 만들기』 — 하네스의 HOW.
- 『하네스 엔지니어링 실전 플레이북』 (본서) — 하네스의 MEASURE.

E-4. 일반 관행

- SRE Post-mortem 템플릿의 4칸 구조 (사건·원인·해결·방지) — Ch8 실패 로그.
- SOLID 원칙 중 Single Responsibility Principle — 에이전트 분리의 일반 원리.

이 책에서 인용된 모든 원출처는 원문 그대로의 문장으로 사용되었거나 요약 표기로 밝혀져 있습니다. 저자가 하지 않은 말을 저자에게 귀속시킨 부분은 없습니다. 만약 발견되면 이 책의 공식 이슈 트래커로 지적해 주시기 바랍니다.